

Test cases and its design Techniques:

- 🔊 Introduction to test case and types
- 🔊 Test case template
- 🔊 How to write a test case and examples
- 🔊 Preparing Review Report
- 🔊 Software Test Life cycle
 - ♣ Writing Test Plan
 - ♣ Preparing Traceability Matrix
 - ♣ Writing Test Execution Report and Summary Report

🔊 Introduction to test case

After Software Development, we need to check each functionality in the product to make sure that actual requirements are fulfilled or not.

A **test case** is a set of conditions or actions that are performed on the software application to verify the expected functionality or feature of application is working as per business requirement.

A Test Case is a script, program, a sequence of steps to validate whether a software application is working bugs free. A well-written test case should allow any tester to understand and execute the test. A collection of test cases is called a **Test Suite**. Different test suites might exercise different components or different types of functionality.

The benefits of an effective test case:

- To validate specific features and functions of the software.
- To help detect usability issues and design gaps early on.
- Reduced maintenance and software support costs.
- Reusable test cases.
- Help verify that the software meets the end user requirements
- Improved quality of software and user experience.

🔊 Test Case types:

Types of test cases can be defined as the variation on the usage of the test cases on each of testing stages in the software testing life cycle.

1. Functional Test Cases
2. Performance Test Cases
3. Security Test Cases
4. Integration Test Cases
5. Positive Test Cases
6. Negative Test Cases
7. Database Test Cases
8. Acceptance Test Cases
9. Usability Test Cases

Test Case Type	Description	Test Step	Expected Result	Status
Functionality	Area should accommodate up to 20 characters	Input up to 20 characters	All 20 characters in the request should be appropriate	Pass or Fail
Security	Verify password rules are working	Create a new password in accordance with rules	The user's password will be accepted if it adheres to the rules	Pass or Fail
Usability	Ensure all links are working properly	Have users click on various links on the page	Links will take users to another web page according to the on-page URL	Pass or Fail

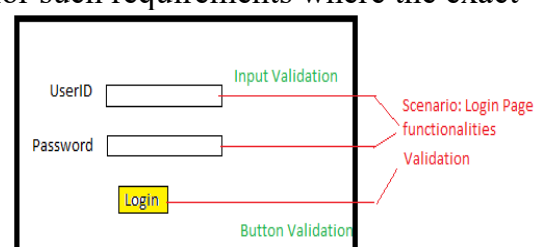
There are two types of test cases:

1. **Formal test cases:** Formal test cases are those test cases which are authored as per the test case format. It has all the information like preconditions, input data, output data, post conditions, etc. It has a defined set of inputs which will provide the expected output.
2. **Informal test cases:** Informal test cases are authored for such requirements where the exact input and output are not known.

Ex: Write test cases for verify login of Gmail account.

Here are some test cases.

1. Enter valid User Name and valid Password
2. Enter valid User Name and invalid Password
3. Enter invalid User Name and valid Password
4. Enter invalid User Name and invalid Password



Test Case Template

A Test Case Template is a well-designed document for developing and better understanding of the test case data for a particular test case scenario. This template is also known as a test case template.

A good Test Case template helps understanding the test cases. Writing test case in a standard format reduces the test effort and the error rate.

Header

Test Case Name/ID :- **Release - Version - Application Name - Module**

Test Case Type:- E.T.C I.T.C S.T.C

Requirement Number:-

Module:-

Severity:- **Critical/Major/Minor**

Status:-

Release:-

Version:-

Pre-condition:-

Test Data:-

Summary:-

Body

Step No.	Description	Inputs	Expected Result	Actual Result	Status	Comments
...	...	...	...	...	...	...
...	...	...	...	...	...	...

Footer

Author:-

Reviewed By:-

Date:-

Approved By:-

Test Case Field	Description
Project Name:	Name of the project the test cases belong to
Module Name:	Name of the module the test cases belong to
Test case ID:	Each test case should be represented by a unique ID. To indicate test types like "TC UI 1" indicating "User Interface Test Case#1."
Name or Test Title/Test Case	Title of the test case
Test Scenario:	Test Scenario ID or title of the test scenario.
Test Priority:	Low, Medium, High
Name of the Module:	Determine the name of the main module or sub-module being tested
Test Designed by:	Name of the Tester who designed the test cases
Date of test designed:	Date When the test cases were created
Test Executed by:	Name of the Tester who executed the test case
Date of the Test Execution:	When the test case was executed
Date Of Review:	When the test cases were reviewed
Description/Summary of Test	Determine the summary or test purpose in brief
Pre-condition:	Conditions which needs to meet before executing the test case.
Reference Document:	Path of the reference documents such as Requirement Document, Test Plan , Test Scenarios etc.
Test Steps:	Procedure to execute the test. describe the steps to perform tests
Test Data:	Data required for execution of test. Data used an input for the test cases. That data needs to be prepared for testing
Expected Results:	The expected output of the test
Post-Condition:	Conditions which needs to achieve when the test case was successfully executed.
Actual Result:	After test execution, The actual output of the test
Status (Fail/Pass)	If the actual and expected results are same, mention it as Passed else Failed. If a test fails, it has to go through the bug life cycle to be fixed.
Test Environment	The environment (H/w, S/w, N/w) in which the test was executed.

Example: write test cases for Verify the login functionality of the Gmail account.

Username

 Password

[Register](#) | [Lost your password?](#)

Here are some test cases.

1. Enter valid User Name and valid Password
2. Enter valid User Name and invalid Password
3. Enter invalid User Name and valid Password
4. Enter invalid User Name and invalid Password

Each test case should analyze to determine how a particular condition is satisfied or not. If the condition is satisfied the Test Status is Passed, Else Failed.

Project Name:	Google Email	 www.SoftwareTestingMaterial.com
Module Name:	Login	
Reference Document:	If any	
Created by:	Rajkumar	
Date of creation:	DD-MMM-YY	
Date of review:	DD-MMM-YY	

TEST CASE ID	TEST SCENARIO	TEST CASE	PRE-CONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POST CONDITION	ACTUAL RESULT	STATUS (PASS/FAIL)
TC_LOGIN_001	Verify the login of Gmail	Enter valid User Name and valid Password	1. Need a valid Gmail Account to do login	1. Enter User Name 2. Enter Password 3. Click "Login" button	<Valid User Name> <Valid Password>	Successful login	Gmail inbox is shown		
TC_LOGIN_001	Verify the login of Gmail	Enter valid User Name and invalid Password	1. Need a valid Gmail Account to do login	1. Enter User Name 2. Enter Password 3. Click "Login" button	<Valid User Name> <Invalid Password>	A message "The email and password you entered don't match" is shown			
TC_LOGIN_001	Verify the login of Gmail	Enter invalid User Name and valid Password	1. Need a valid Gmail Account to do login	1. Enter User Name 2. Enter Password 3. Click "Login" button	<Invalid User Name> <Valid Password>	A message "The email and password you entered don't match" is shown			
TC_LOGIN_001	Verify the login of Gmail	Enter invalid User Name and invalid Password	1. Need a valid Gmail Account to do login	1. Enter User Name 2. Enter Password 3. Click "Login" button	<Invalid User Name> <Invalid Password>	A message "The email and password you entered don't match" is shown			

How to write a Test Case

Two ways of writing test case through which tester can verify the functionalities of the software.

- Positive Test Cases: to verify the application behavior on positive test data input.
- Negative Test Cases: to verify the system behavior on negative or invalid test data input.

Write good and effective test cases follow some steps:

1. Identify the scope and purpose of testing

You have to need understand the purpose of testing, features and user requirements. You need to identify the testable requirements

2. Define how to perform testing activities

You should have better understanding of functional and non-requirements of an application. You must know about what all operations can be used, test scenarios & test cases can be written based on requirements.

3. Avoid test case repetition.

Do not repeat test cases. If a test case is needed for executing some other test case, call the test case by its test case id in the pre-condition column. Use unique Test Case ID

4. Define the test case framework

Test case framework can be designed depend on systems requirements. Test cases should cover functionality, compatibility, UI interface, fault tolerance, and performance.

5. Preparation of test data for variety of test cases

Variety of test data is required for executing the test cases like valid data to test positive test cases, invalid data to test negative test cases, legal invalid data to test boundary value conditions, illegal and abnormal data to test error handling and recovery test cases

6. Structuring of test cases

Required all the necessary information to writing test cases. We have detailed preparation list of test scenarios from the requirements. Each test scenario has one or multiple test cases.

7. Test Cases need to be simple, transparent and easy to understand.

Test cases should be detailed and clear, new tester can easily execute the test cases. The purpose and scope of test cases should be well defined.

8. Test cases should be valid, brief and short

Test cases should have only necessary and valid steps. If single test case having too many test steps to be performed then this may lose concentration & aim of test case. So, single test case should have one expected result and not try to cover too many expected results.

9. Test Cases Should Be Traceable

Each and every test case should be traceable and it should be linked to “Requirement ID”, which helps to check the 100% coverage of requirements. This will help to identify the tester is performing testing for all requirements. No any requirement is missed to test.

10. Test cases should be reusable & maintainable:

A well written test case is reusable and maintainable. There are times where developers change the code, and testers need to update the test cases. If our test cases are easy to read and understand then it would be easy to update them not only by whoever wrote it but also by other testers as well.

➤ Write Test Cases for Registration and Login Page

You should gather all the customer requirements you can. The list of requirements which can be useful are:-

Registration Form having following fields:-

- | | | | |
|--------------|---------------------|-------------|----------------|
| 1. FirstName | 2. LastName | 3. UserName | 4. Email Id |
| 5. Password | 6. Confirm Password | 7. Address | 8. PhoneNumber |
| 9. Gender | | | |

Required fields are: 1. EmailId 2. UserName 3. Password 4. Confirm Password

Validation for fields:

1. Email
2. Password: It should have alphanumeric; Length should be 8 to 32
3. PhoneNumber: Phone Number should have only Numbers, Country code is required

— After successful Verification email should send to the user

➤ Test case for registration page

Signup page and login page by assuming some client requirements such as:-

- Username and password are mandatory fields
- There is a cancel and reset button at the bottom of the form
- Radio buttons and checkboxes are placed correctly
- The limit of Password should be 8-13 characters (alphanumeric).

Sr. No	Test Cases	Unit / Module	Description	Test Steps	Test Data/Input	Expected Results
1	TC-001	User Interface	Check all the text boxes, radio buttons, etc	1. Click on Radio buttons, buttons and dropdowns		UI should be perfect
2	TC-002	Required fields	Check the required fields by not filling any data	1. Do not enter any value in the field 2. Click on Signup button	Nil	It Should show mandatory symbol (*) on mandatory fields.
3	TC-003	Required fields	Check the required fields by filling the data	1. Enter valid values in required fields 2. Click on Signup button	Nil	1. User should be register successfully 2. A successful registration message should show 3. Mail should send to the user

Unit-3 Test cases design Techniques

4	TC-004	Optional fields	Check all the optional field when do not fill data	1. Do not enter any detail in optional fields 2. Enter valid data in required fields 3. Click on Signup button	Nil	1. It should not ask to fill the optional fields 2. User should be register successfully 3. A successful registration message should show 4. Mail should send to the user
5	TC-005	Optional fields	Check all the optional field when fill data	1. Enter valid data in optional fields 2. Enter valid data in required fields 3. Click on Signup button	Nil	1. User should be register successfully 2. A successful registration message should show 3. Mail should send to the user
6	TC-006	Email validation	— Check the Email text field that has Email address without @ symbol. — Check the Email text field that has random string instead of real email. — Check the Email text field that has @ symbol written in words. — Check the Email text field that has missing dot in the email address.	1. Enter Invalid Emails 2. Click on Signup Button	1. testAtgmail.com 2. test@gmailcom 3. test@gmail 4. @gmail	It should show the validation message for valid email
7	TC-007	Email validation	Check all the valid emails	1. Enter Valid Emails 2. Click on Signup Button	1.test@192.168.1.1 2. test@gmail.com	It should not show any validation message
8	TC-008	Phone Number validation	Check the phone number when pass alphanumeric data	1. Enter alphanumeric data in phone field 2. Click on signup button	1. dada5\$7567#7	It should show the validation message for Phone Number
9	TC-009	Phone Number validation	Check the phone number when not pass country code	1. Enter valid phone number without country code 2. Click on signup button	1. 9012078654	It should show the validation message for country code is required
10	TC-010	Phone Number validation	Check the phone number when pass country code	1. Enter valid phone number with country code 2. Click on signup button	1. +9190112244	It should not show any validation message
11	TC-011	Password validation	Check the password limit when enter value less than min	1. Enter value which is alphanumeric but less than 8 2. Click on signup button	1. Password	It should show validation message
12	TC-012	Password validation	Check the password limit when enter value greater than max	1. Enter value which is alphanumeric but less more than 32 2. Click on signup button	1. redhatisgood434ad asdasdasd3243dsad adsad432423	It should show validation message
13	TC-013	Password validation	Check the password when pass only numbers	1. Enter value in numbers which is in between 8-32 2. Click on signup button	1. 12345678	It should show validation message
14	TC-014	Password validation	Check the password when pass valid data	1. Enter value in alphanumeric which is in between 8-32 2. Click on signup button	1. Pass123456	It should not show any validation message Test case for Login page

Test case for login page

Sr. No.	Test Cases id	Module	Test Case	Test Steps	Expected Results
1	TC-001	User Interface	Check all the text boxes, and buttons	Check page	UI should be perfect. Text boxes and button should be aligned
2	TC-002	User Login	Check When pass invalid user name	1. Enter invalid username 2. Enter correct password 3. Click on Login Button	User should not login and should show proper error message
3	TC-003	User Login	Check When pass correct username and invalid password	1. Enter valid username 2. Enter incorrect password 3. Click on Login Button	User should not login and should show proper error message
4	TC-004	User Login	Check Keeping Password blank	1. Enter valid username 2. Do not enter password 3. Click on Login Button	User should not login and should show proper error message
5	TC-005	User Login	Check when pass correct email and password	1. Enter valid username 2. Enter valid password 3. Click on Login Button	— User should login
6	TC-006	User Login	Check if password is entered in encrypted form	1. Enter valid username 2. Enter password 3. Click on Login Button	Password is entered in encrypted form
7	TC-007	Signup for new users	Check whether the signup link for new user is working	Click Signup link	Clicking signup link takes user to signup page successfully

Preparing Review Report

Review is generally performed by:

1. **Self-review:** It is done by the tester himself who has written the test cases. He can verify whether all the requirements are covered or not by looking into SRS/FRD.
2. **Peer review:** It is done by another tester who hasn't written those test cases but is familiar with the system under test. Also known as Maker and Checker review.
3. **Review by a supervisor:** It is done by a team lead or manager who is superior to the tester who has written the test cases and has great knowledge about the requirements and system under test.

Test case review processes are:

1. Spelling mistakes: it can create a lot of confusions or make a sentence difficult to understand.
2. Grammar: If grammar of test cases is not proper then interpreting may be wrong results.
3. Template format: Proper template becomes easy to add/modify test cases in future and test case plan looks organized.
4. Standard/Guidelines: While review process to check whether all the standards and guideline are properly followed.
5. Language used: Test cases should have a very simple language which is easy to understand.
6. Functionality coverage: all functionality associated with the system under test should be covered
7. Replication: It refers to the duplicate test cases removal. It is possible that two or more test cases test the same. They can merged into one, this would save time and space.

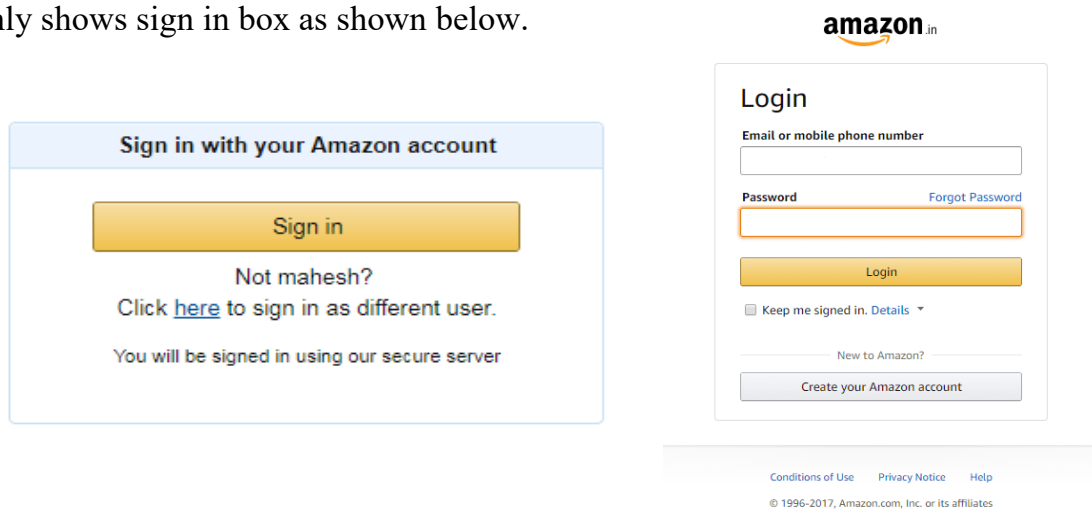
Sample Template for Test Case Review

Test Case ID: _____ Reviewed By: _____ Date of Review: _____ Version of Review: _____				Review Comments Incorporated By : _____ Date Incorporated: _____ New Version of Document: _____		
Sr.	Page No.	Step No.	Severity C – Critical, M – Major, S – Small	Comments	Action Taken	Close (Yes / No)
01						
02						
03						
04						
05						
06						
07						
08						
09						
10						

Examples On Writing Test Cases

2) Amazon Login Scenario

When you already login to the Amazon and revisit the website. It will show you special box that shows you are already identified. And it requires you to enter password to next page. And for that flow, it only shows sign in box as shown below.



The image shows two versions of the Amazon login interface. On the left is a simplified version with a blue header 'Sign in with your Amazon account', a yellow 'Sign in' button, a link 'Not mahesh? Click here to sign in as different user.', and a note 'You will be signed in using our secure server'. On the right is the actual Amazon.in login page, featuring the Amazon logo, a 'Login' header, input fields for 'Email or mobile phone number' and 'Password', a 'Forgot Password' link, a yellow 'Login' button, a 'Keep me signed in' checkbox with a 'Details' dropdown, a 'New to Amazon?' section with a 'Create your Amazon account' button, and footer links for 'Conditions of Use', 'Privacy Notice', and 'Help'.

If the user has not logged in before then it shows you username and the password through the flow. It takes username at first and then password is added on the next part.

Positive Test Cases

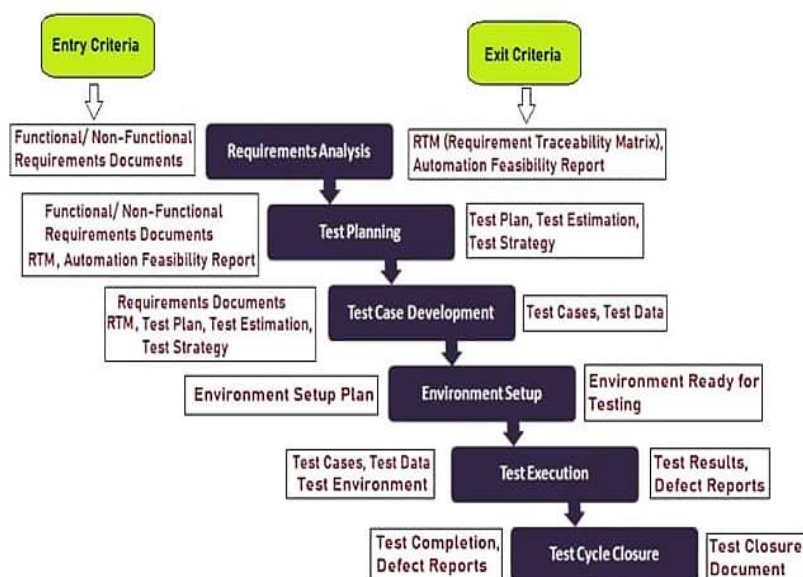
- Enter Valid Username.
- Enter Valid Password.
- Click on “Forgot Password” Link and find out forgot password page.
- Click on Create new account button and verify if it takes to new account page.
- Select “Keep me signed in” option to retain login session.

Negative Test Cases

- Enter invalid username and valid password.
- Enter valid username and invalid password.
- Enter invalid username and password.
- Keep one field blank either username or password and check login.

Software Test Life Cycle (STLC)

It is the sequence of activities carried out by the [testing team](#) from the beginning of the project till the end of the project. Each activity is carried out in a planned and systematic manner in order to meet the quality goals. In the STLC process, each phase has different goals and deliverables. The different phases of Software testing life cycle are:



Each of the steps mentioned

- **Entry Criteria:** It defines the precondition that must be completed before testing can begin
- **Exit Criteria:** defines conditions that must be fulfilled before testing can be stop.

1. Requirement Analysis

This is the very first phase of Software testing Life cycle (STLC).

- In this phase testing team goes through the Requirement document with both Functional and non-functional details to identify whether the requirements are testable or not.
- If anything is missing or not understandable then quality assurance team meets with the stakeholders to better understand the detail knowledge of requirement.
- Once the QA team is clear with the requirements they will document the acceptance Criteria and get it approved by the Customers.
- Testers analyze customer requirements and work with developers during the design phase to see which requirements are testable and how they are going to test those requirements.
- To start testing activities from the requirements phase because the defectfixing cost is very less if it is found in requirements phase rather than in next phases.

Activities to be done in Requirement analysis phase are given below:

- Identify types of tests to be performed.
- Identify test priorities
- Prepare RTM (Requirement Traceability Matrix)
- Identify details about the test environment
- Automation Feasibility Analysis.

Entry Criteria: BRS (Business Requirement Specification) Availability of requirement Document, acceptance criteria and application architectural document.

Deliverables (Outcome) of Requirement analysis:

- Requirement Traceability Matrix (RTM)
- Automation feasibility report

2. Test Planning and Control (Test Strategy)

Test Planning phase starts after the completion of the Requirement Analysis phase.

- In this phase QA manager prepare and finalize test plan based on the requirement analysis.
- QA manager will analyze cost or efforts for the project.
- In this phase all the testing planning is done like what needs to be tested, how the testing will be done, test strategy to be followed, what will be the test environment, what is testing methodologies will be followed, hardware and software availability, resources, risks etc.

Activities to be done in Test Planning phase are given below:

- Estimation of testing effort
- Selection of Testing Approach
- Preparation of Test Plan, Test strategy documents
- Resource planning and assigning roles and responsibility to them
- Selection of Testing tool

Entry Criteria: requirements documents, requirement traceability matrix, test automation feasibility document

Deliverables (Outcome) of Test Planning phase:

- Test Plan document
- Test Strategy document
- Number of Resources, skill required and their roles and responsibilities
- Testing tool to be used

3. Test Case Development

- The test case development activity is started once the test planning activity is finished. testing team write down the detailed test cases.
- Each case defines test inputs (e.g. data), procedures, execution conditions, and anticipated results.

- Test cases should be transparent, efficient, and adaptable. Ensure 100% coverage of software requirements. Implement testing techniques
- Once all test cases are ready then these test cases are reviewed by peer members or QA lead.
- The Requirement Traceability Matrix is a format for tracking requirements where each test case is mapped with the requirement. Using this RTM we can track backward & forward traceability.

Activities to be done in Test Case Development phase are given below:

- Creation of test cases and test scripts
- Verification of test cases and automation scripts
- Creation of Test Data in testing environment

Deliverables (Outcome) of Test Case Development phase are:

- Test cases
- Test scripts (if required)
- Test Data

4. Test Environment setup

- Team decides how they will perform the tests. They define test conditions, setting up the test environment. The development team or customer provides the test environment.
- Once the environments are successfully deployed, smoke testing can be performed to ensure the environment is working with the intended functionality and would also provide confidence in testing the new feature. This activity can be started parallel with Test Case Development independently.
- This phase includes the setup or installation process of software and hardware which is required for testing the application.
- After setting up the required software and hardware the installation of build is tested. Once the installation of build is successful and complete then the Test Data is generated.

Activities to be done in Test Environment Setup phase are given below:

- As per the Requirement and Architecture document the list of required software and hardware is prepared
- Setting up of test environment
- Creation of test data
- Installation of build and execution of Smoke testing on it

Deliverables (Outcome) of Test Environment Setup phase are:

- Test Environment setup is ready
- Test Data is created
- Results of Smoke testing

5. Test Execution

- Before starting the Test Execution phase the Test Case Development and Test Environment setup should be ready.
- In Test Execution phase the test cases are executed in the testing environment.
- In this phase testing team start executing test cases based on prepared test planning & prepared test cases in the prior step.
- While execution of the test cases the QA team may find bugs which will be reported against that test case. This bug is fixed by the developer and is retested by the QA.
- Once the test case is passed then same can be marked as Passed. If any test case is failed then corresponding defect can be reported to developer team via bug tracking system & bug can be linked for corresponding test case for further analysis.
- Every failed test case should be associated with at least single bug. Once the bug fixed by development team then same test case can be executed based on your test planning.
- If any of the test cases are blocked due to any defect then such test cases can be marked as

Blocked, so we can get the report based on how many test cases passed, failed, blocked or not run etc. Once the defects are fixed, same Failed or Blocked test cases can be executed again to retest the functionality.

Activities to be done in Test Execution phase are given below:

- Execution of Test Cases
- Reporting test results
- Logging defects for the failed test cases
- Verification and retesting of the defect
- Closure of defects

Deliverables (Outcome) of Test Execution phase are:

- Test execution Report
- Updated test cases with results
- Bug Report

6. Test Cycle Closure

- In order to start the Test Cycle Closure activity the Test Execution phase should be completed. In Test Cycle phase the QA team will meet and discuss about the testing piece.
- The whole goal of this discussion is to learn lessons from the bad practices. This will help in future projects.
- Call out the testing team member meeting & evaluate cycle completion criteria based on Test coverage, Quality, Cost, Time, Critical Business Objectives, and Software.
- Discuss what all went good, which area needs to be improve & taking the lessons from current STLC as input to upcoming test cycles, which will help to improve bottleneck in the STLC process.
- Test case & bug report will analyze to find out the defect distribution by type and severity.
- Once complete the test cycle then test closure report & Test metrics will be prepared. Test result analysis to find out the defect distribution by type and severity.

Activities to be done in Test Cycle Closure phase are given below:

- To evaluate the test completion on the basis of Test Coverage and Software Quality
- Documentation of the learning from the project
- Analyzing the test results to find out the distribution of severe defects
- Test Closure Report preparation

Deliverables (Outcome) of Test Cycle Closure phase are:

- Report of Test Closure

STLC Phase	Entry Criteria	Exit Criteria	Deliverables
Requirement Analysis	Requirements specification document, Acceptance criteria document, Application architectural document	Signed off RTM, Signed off Automation feasibility report	List of all testable requirements, Automation feasibility report
Test Planning	Requirements Documents, Automation feasibility report	Approved Test plan document, Approved Test strategy document, Signed off effort estimation document	Test Strategy, Test Plan, Test Effort estimation document.
Test Design	Requirements Documents (Updated version of unclear or missing requirement), RTM, Test Plan, Test Estimation Document, Automation Analysis Report	Reviewed and approved test cases, test scripts, test data	Test cases, Test Scripts, Test data.
Test Environment Setup	Test Plan, Test environment setup plan, Smoke Test cases, Test Data	Working test environment setup, Valid test data setup, Successful smoke test	Test Environment. Smoke Test Results.

Test Execution	Test Plan document, Test cases, Test Scripts, Test data, Test Environment.	Execute all planned test cases, Log all defects found	Test case execution report, Defect reports, RTM
Test Closure	Testing has been completed, Test Case Execution report (make sure there are no high severity defects opened)	Signed off Test Closure report	Test Closure report, Test metrics

📣 Writing Test Plan

A test plan is a detailed document that outlines the test strategy, [Testing](#) objectives, resources (manpower, software, hardware) required for testing, test schedule, [Test Estimation](#) and test deliverables.

The test plan provide a blueprint to conduct software testing activities process which is monitored and controlled by the test manager. Test plans outline the process of testing the functionality of software. A test plan details each step taken to achieve a certain result and states the objective of each action. The plan also highlights the projected resources, risks, and personnel involved in the test. You should use a test plan if you are seeking to eliminate bugs and other errors in your software before it becomes available to customers.

➤ Test Plan types

- **Master Test Plan:** A high-level plan, which includes all others.
- **Testing Level Specific Test Plans:** Plans for different testing levels- Unit testing, Integration testing, System testing, Acceptance testing
- **Testing Type Specific Test Plans:** Plans for types of testing as Performance Test Plan and Test Plan Security.

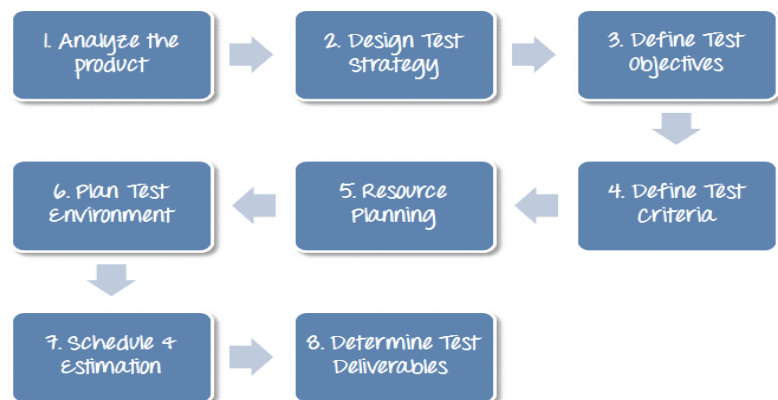
➤ Importance of Test Plan

Making Test Plan has multiple benefits

- Test Plan helps us to determine the effort needed to validate quality of the application under test
- Helps to developers, business managers, customers understand the details of testing.
- Test Plan guides our thinking. It is like a rule book, which needs to be followed.
- Test estimation, test scope, Test Strategy are documented in Test Plan, so it can be reviewed by Management Team and re-used for other projects.

➤ How to write a Test Plan

1. Analyze the product
2. Design the Test Strategy
3. Define the Test Objectives
4. Define Test Criteria
5. Resource Planning
6. Plan Test Environment
7. Schedule & Estimation
8. Determine Test Deliverables



🕒 Step 1) Analyze the product

How can you test a product without any information it is Impossible. You must learn a product carefully before testing it. You should research clients and the end users to know their needs and expectations from the application

- Who will use the website?
- What is it used for?
- How will it work?
- What are software/ hardware the product uses?

🕒 Step 2) Develop Test Strategy

A Test Strategy document is a high-level document, which is usually developed by Test Manager.

This document defines:

- The project's testing objectives
- Determines testing effort and costs



Step 2.1) Define Scope of Testing: Before starting any test activity, scope of the testing should be known.

1) In Scope: Scope defines the features, functional or non-functional requirements of the software that will be tested. The components of the system to be tested (h/w, s/w, middleware etc)

2) Out of Scope: Out Of Scope defines the features, functional or non-functional requirements of the software that will not be tested. The components of the system that will not be tested also need to be clearly defined

Defining accurate scope of testing helps you-

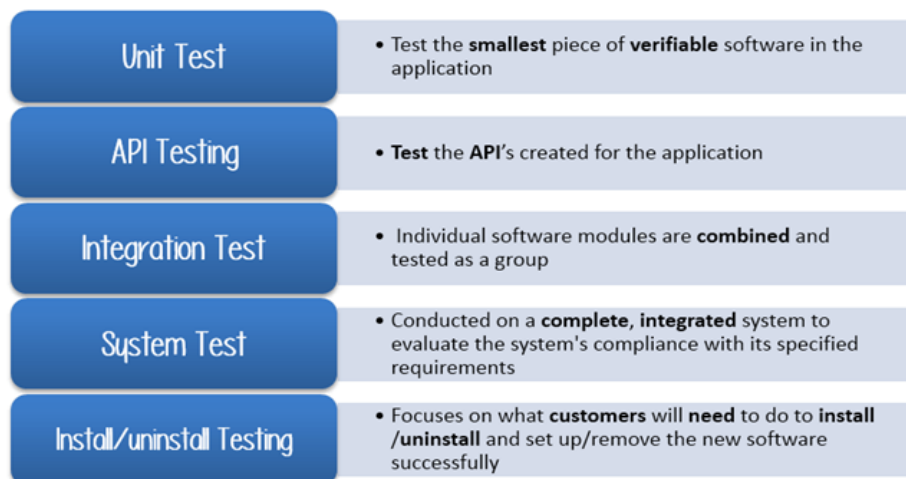
- Give everyone a confidence & accurate information of the testing you are doing
- All project members will have a clear understanding about what is tested and what is not

How do you determine scope your project?

- accurate customer requirement
- Project Budget
- Product Specification
- Skills & talent of your test team

You should clearly define the "in scope" and "out of scope" of the testing.

Step 2.2) Identify Testing Type: A Testing Type is a standard test procedure that gives an expected test results. Each testing type is planned to identify a specific type of product bugs. But, all Testing Types aim is achieving one common goal “Early bug detection before releasing the product to the customer”



There are number of **Testing Types** but your testing team **cannot have** enough efforts to handle all kind of testing. As Test Manager, you must set **priority** of the Testing Types

- Which Testing Types should be **focused** for web application testing?
- Which Testing Types should be **ignored** for saving cost?

Step 2.3) Document Risk and Issues (Risk Management): Risk is future's uncertain event with a probability of loss. When risk actually occurs, it becomes 'issue'.

List out the risks involved in testing process. Along with risks write mitigation & contingency actions if risk occurred.

Some basic rules to follow at the time of developing test plan:

- Keep your plan flexible so that it can be enhanced later for adding test cases, test data etc.
- Change control mechanism should be there to track changes.

- Test plan should be reviewed frequently before base lining.
- Test plan should be short, readable because test plan act as a communication document.

Risk	Mitigation
Team member have lack of skills for website testing.	Plan training course to skill up your members
The project schedule is too tight; it's hard to complete this project on time	Set Test Priority for each of the test activity.
Test Manager has poor management skill	Plan leadership training for manager
A lack of cooperation negatively affects your employees' productivity	Encourage each team member in his task, and inspire them to greater efforts.

Step 2.4) Create Test Logistics: Test Manager should answer the following questions:

Who will test?

To select the right member for specified task if his skill is qualified for the task or not, also estimate the project budget. Type of tester can be defined. Selecting wrong member for the task may cause the project to **fail** or **delay**.

Person having the following skills:

- Ability to **understand** customers point of view
- Strong **desire** for quality
- **Attention** to detail
- Good **cooperation**

When will the test occur?

Test activities must be matched with associated development activities. You will start to test when you have **all required items** shown in following figure



◎ Step 3) Define Test Objective

Test Objective is the overall goal and achievement of the test execution. The objective of the testing is finding as many software defects and ensure that the software is **bug free** before release.

To define the test objectives:

1. List all the software features (functionality, performance, GUI) which may need to test.
2. Define the target or the goal of the test based features
3. Ensure the Application Under Test conforms to functional and non-functional requirements
4. Ensure the AUT meets the quality specifications defined by the client
5. Bugs/issues are identified and fixed before go live

◎ Step 4) Define Test Criteria

Test Criteria is a standard or rule on which a test procedure or test judgment can be based.

Two types of test criteria:

1. Test Suspension & Resumption Criteria:

Test suspension criteria should explain when to pause/suspend the testing activities.

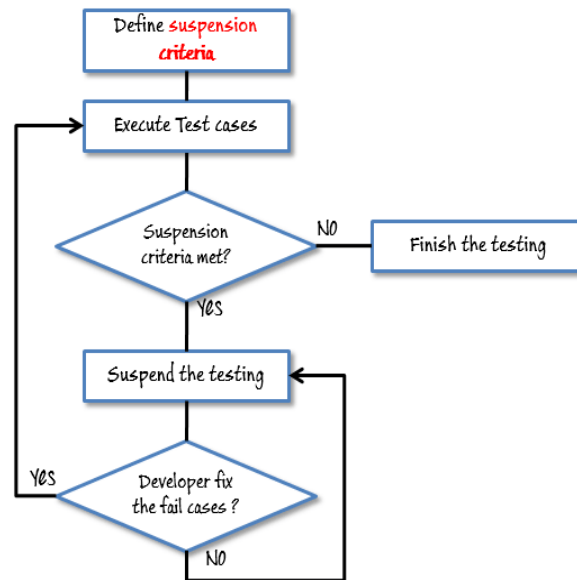
Test resumption criteria explains when to continue with testing activities after resolving issues.

Test team need to suspend or terminated the testing because of some major enhancements in project. If the suspension criteria are met during testing, the active test cycle will be **suspended** until the criteria are **resolved**.

Example: In your team members report shows there are **40%** of test cases failed, you should **suspend** testing until the development team fixes all the failed cases.

2. Test Exit Criteria

A test exit criterion expresses when to stop testing. Factors like release deadlines, test coverage, test budget reduction, confidence in test executed. Example: **95%** of all critical test cases must be passed.



Some methods of defining exit criteria are

- **Run rate** is ratio between **number test cases executed/total test cases** of test specification.
- Example: total 120 TCs, but the tester only executed 100 TCs, so the run rate is $100/120 = 0.83$ (83%)
- **Pass rate** is ratio between **numbers test cases passed / test cases executed**.
- Example: 100 TCs executed, there are 80 TCs passed, so the pass rate is $80/100 = 0.8$ (80%)

Your Team reported test execution result to you and they want to confirm the **Exit Criteria**. In case, the Run rate is mandatory is **100%**, but the test team only completed 90% of test cases. It means the Run rate is not satisfied, so do not confirm the Exit Criteria.

◎ Step 5) Resource Planning

Resource plan is a **detailed summary** of all resource types required to complete project task. Resource could be human, equipment and materials needed to complete a project. The resource planning is important factor of the test planning because helps in determining the number of resources (employee, equipment etc.) to be used for the project. Test Manager can make the correct schedule & estimation for the project. Detail description of the Roles and responsibilities of different team members are for each stage of testing activity.

1. Human Resource: The following table represents various members in your project team

Member	Tasks
Test Manager	Test Managers responsibility is to plan & manage testing activities, preparing reports etc. Deliverables includes test plan, test cases/Scripts, defect logs, reports, project directions
Tester	Tester's responsibility is to execute tests, defect logging & tracking.
Test Administrator	Builds up and ensures Test Environment and resources are managed and maintained. Support Tester to use the test environment for test execution
SQA members	Check to confirm whether the testing process is meeting specified requirements

2. System Resource

Resources	Descriptions
Server	web server, database server, and application server
Test tool	test tools are Selenium, QTP etc.
Network	Network devices include LAN and Internet
Computer	The PC which users often use to connect the web server

◎ Step 6) Plan Test Environment

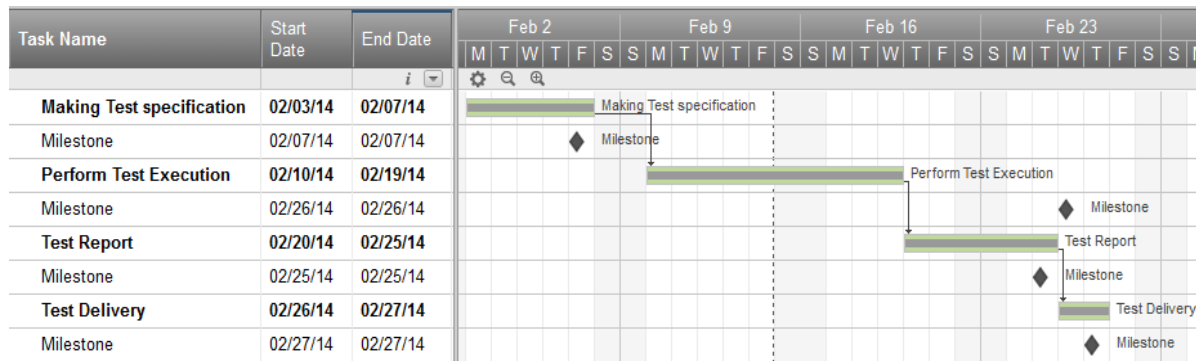
A testing environment is a setup of software and hardware on which the testing team is going to execute test cases. Test environment should describe how to set up test environment, resources like software, hardware, network and tools like test management tools, configuration & version management tools, automated testing tools, defect tracking tools.

◎ Step 7) Schedule & Estimation

Schedule should contain a tasks with planned start & end dates. It should include tasks like, test plan preparation & approval, creation of test cases & reviews, execution of test cases, test summary report etc. you should include that estimation as well as the schedule to the Test Planning.

Task	Members	Estimate effort
Create the test specification	Test Designer	170 man-hour
Perform Test Execution	Tester, Test Administrator	80 man-hour
Test Report	Tester	10 man-hour
Test Delivery		20 man-hour
Total		280 man-hour

Example: Suppose the boss wants to complete the project in **one** month, you already estimated the effort for each tasks in Test Estimation. You can create the schedule as:



◎ Step 8) Test Deliverables

Test Deliverables is a list of all the documents, tools and other components that has to be developed and maintained in support of the testing effort. There are different test deliverables at every phase of the software development lifecycle.



Test deliverables are provided **before testing phase.**

- Test plans document.
- Test cases documents
- Test Design specifications.

Test deliverables are provided **during the testing**

- Test Scripts
- Test Data
- Test Traceability Matrix
- Error logs and execution logs.

Test deliverables are provided **after the testing** cycles is over.

- Test Results/reports
- Defect Report
- Installation/ Test procedures guidelines

➤ Test Plan Template

The format and content of a software test plan vary depending on the processes, standards, and test management tools being implemented. Following format, which is based on IEEE standard for software test documentation, provides a summary of what a test plan can/should contain.

◆ Test Plan Identifier:

- Provide a unique identifier for the document.

◆ Introduction:

- Provide an overview of the test plan.
- Specify the goals/objectives.
- Specify any constraints.

◆ References:

- List the related documents, with links to them if available, including the following:
 - Project Plan
 - Configuration Management Plan

◆ Test Items:

- List the test items (software/products) and their versions.

◆ Features to be Tested:

- List the features of the software/product to be tested.

◆ Features Not to Be Tested:

- List the features of the software/product which will not be tested.
- Specify the reasons these features won't be tested.

◆ Approach:

- Mention the overall approach to testing.
- Specify the testing levels, the testing types, and the testing methods [Manual/Automated; White Box/Black Box/Gray Box]

◆ Item Pass/Fail Criteria:

- Specify the criteria that will be used to determine whether each test item has passed or failed.

◆ Suspension Criteria and Resumption Requirements:

- Specify criteria to be used to suspend the testing activity.
- Specify testing activities which must be redone when testing is resumed.

◆ Test Deliverables:

- List test deliverables, and links to them if available, including the following:
 - Test Plan (this document itself)
 - Test Cases
 - Test Scripts
 - Defect/Enhancement Logs
 - Test Reports

◆ Test Environment:

- Specify the properties of test environment: hardware, software, network etc.
- List any testing or related tools.

◆ Estimate:

- Provide a summary of test estimates (cost or effort) and/or provide a link to the detailed estimation.

◆ Schedule:

- Provide a summary of the schedule, specifying key test milestones, and/or provide a link to the detailed schedule.

◆ Staffing and Training Needs:

- Specify staffing needs by role and required skills.
- Identify training that is necessary to provide those skills, if not already acquired.

◆ Responsibilities:

- List the responsibilities of each team/role/individual.

◆ Risks:

- List the risks that have been identified.
- Specify the mitigation plan and the contingency plan for each risk.

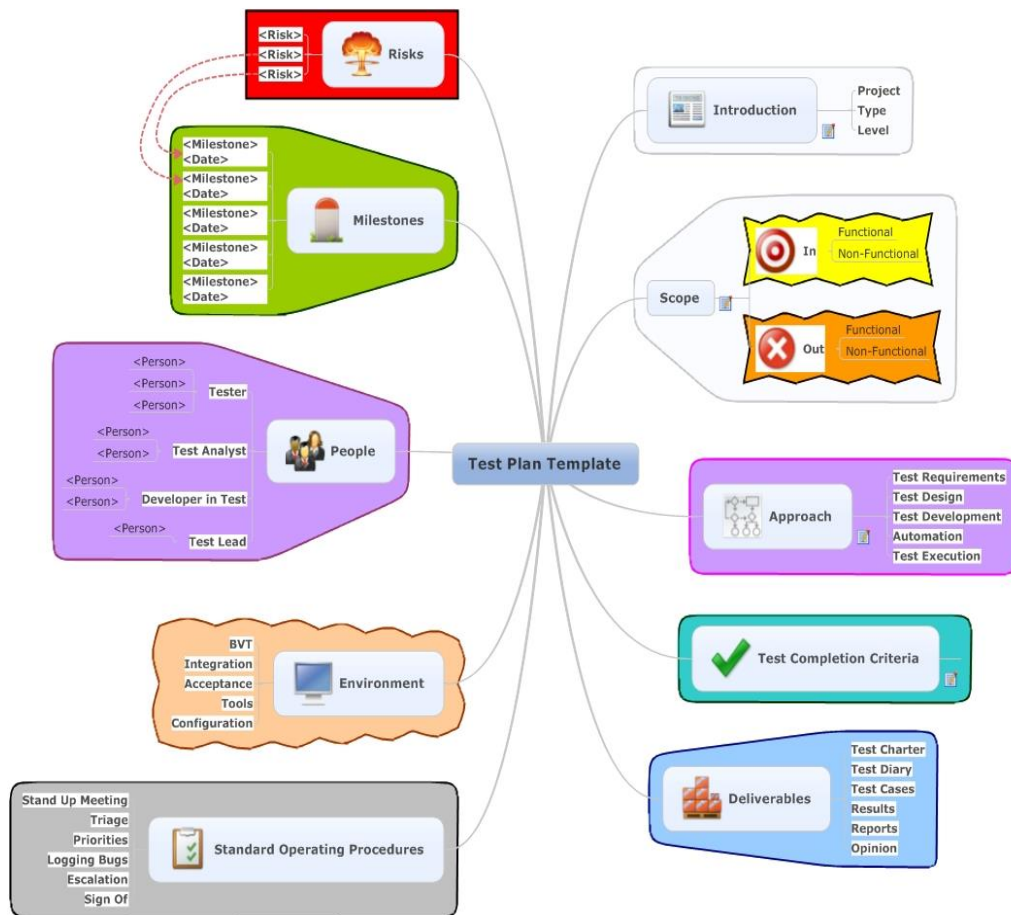
◆ Assumptions and Dependencies:

- List the assumptions that have been made during the preparation of this plan.
- List the dependencies.

◆ Approvals:

- Specify the names and roles of all persons who must approve the plan.

- Provide space for signatures and dates. (If the document is to be printed.)



Example: A Sample Test Plan Document for Internet Banking Application

1) Test Plan ID: IBS_ST_TP_001

2) Introduction: It is System Test Plan for Internet Banking System, internet web application, provides access to Account holders and guest users from any where in the world. It has two interfaces one is Admin interface another is User interface. Admin can be accessed by Bank authorized users, user interface can be accessed by Bank account holders and guest users.

The purpose of the system (Application) is to provide bank information and services online (through Internet), Bank account holders can get Banking services from any where, without visiting the Bank branches.

3) Test Items:

Admin Interface: Master Data User Management Report etc.	User Interface: Information Personal Banking Corporate Banking Business etc.
--	---

4) References:

- Requirements
- Project Plan
- Test Strategy
- Use cases (If available)
- High Level Design doc
- Low Level design docs
- Process guide line doc
- Prototypes

5) Features to be tested:

a) Admin Interface:

i) Master Data 1) Add new branch, Edit Branch /Delete Branch 2) Add new ATM 3) Add new loan type 4) Add new account type 5) Add new deposit type etc.	ii) User Management 1) Create new user 2) Edit user 3) Delete user etc.	iii) Reports 1) Branch wise report 2) User wise report 3) Day, month, yearly reports 4) Service wise report (only loans, only new account. fixed deposits)
---	---	---

b) User Interface:

i) Information 1) Branch locators 2) ATM locators 3) Loans information 4) Bank history 5) Bank financial details 6) Fixed deposits information	ii) Personal Banking 1)Login 2)Balance enquiry 3)Bill payment (utilities, subscriptions) 4)Fund transfer (transfer to same bank, others banks) 5) Statement generation (mini stmt, detailed report)etc.	iii) Corporate Banking 1) Add user, Edit user, Delete user 2) Balance enquiry 3) Money transfer 4) Payroll 5) Reports etc.
---	---	--

6) Features not to be tested: NA**7) Entry Criteria:**

a) Test Design Team formation, Responsibilities, Schedule, Requirements, Test Case Template etc. Training on Domain, on Automation tools	b) Test Execution: Readiness of Test Lab Readiness of AUT Requirements Test Case docs Test Data Defect Report Template etc.
---	--

8) Exit Criteria:

- All possible test cases executed
- Maximum defects fixed, Final Regression performed successfully
- Confidence on Test process
- Time Limitations
- Budget Limitations

9) Suspension Criteria:

- Show-Stopper bug found
- Supplier issues
- Vast changes in requirements
- If resolving defects are more

10) Roles & Responsibilities

Sno	Name	Role	Responsibilities	Remarks
1	Kumar	Test Lead	Test Planning, guidance, Monitoring and Test control	
2	Venkat Rao	Sr. Tester	Test Data Collection, Generating Test Scenarios.	
3	Swapna	Tester	Test Case Documentation, Test execution, defect reporting and tracking for Admin module.	
4	Srinivas	Tester	Test Case Documentation, Test execution, defect reporting and tracking for Personal Banking module.	
5	Suneetha	Tester	Test Case Documentation, Test execution, defect reporting and tracking for Corporate Banking module.	

11) Schedule:

Sno	Task	Days	Duration	Remarks
1)	Understanding & Analyzing Requirements	5	2 nd July to 6 th July	
2)	Review Meeting	01	9 th July	
3)	Generating Test Scenarios.	10	11 th July to 22 nd July	

Unit-3 Test cases design Techniques

4)	Reviews	02	25 th July to 26 th July	
5)	Test Case Documentation	40	29 th July to 12 th August	
6)	Reviews	04	14 th August to 18 th August	
7)	Test Data collection	06	20 th August to 26 th August	
8)	Reviews	01	28 th August	
9)	Verifying Test Environment setup	01	29 th August	
10)	Create Test batches	02	30 th , 31 st August	
11)	Sanity Testing	01	3 rd September	
12)	Comprehensive Testing	25	4 th September to 2 nd October	
13)	Sanity Testing	01	3 rd October	
14)	Selecting Test cases	02	4 th and 5 th October	
15)	Regression Testing	05	8 th October to 12 th October	
16)	Sanity Testing	01	15 th October	
17)	Selecting Test cases	01	16 th October	
18)	Regression Testing Cycle 2	04	17 th October to 22 nd October	
.	.	.	.	
28)	Final Regression	08	19 th November to 28 th November	
29)	Evaluating exit criteria	1 or 2	29 th , 30 th of November	
30)	Collecting all artifacts	02	3 rd , 4 th December	
31)	Test Summary Report	01	5 th December	

Note: Regression Testing depends on Application and strength of Development team.

12) Training

- Training Program on Banking Domain
- Test Automation Training using QTP Tool

13) Risks & Mitigations

- Team members issues
- Vendor issues
- Time
- Budget

14) Test Environment / Lab

Application Type: Web Application, Internet and Public

- **Server side:**
 - Windows 2003 Server
 - UNIX server
 - Ms Exchange server -a) Web server, b) EDP, c) Data storage
 - Bugzilla Tool
 - VSS
 - MS Office
 - QTP Tool
 - Browser IE 7.0
- **Client side:**
 - Windows XP + SP2
 - VSS
 - MS Office
 - QTP Etc.
- **AUT Environment**
 - .NET (C#, VC++, ADO)
 - IIS -web server
 - COM+ -App server
 - SQL Server 2005 for Database server

15) Test Deliverables

- Test Plan
- Review Reports
- RTM
- Test Scenario docs
- Test Case docs
- Test data
- Opened, Closed Defect Reports
- Test Summary Report

16) Approvals

SNO	Task/s	Author /Role	Date & Signature
1)	Test Plan Documentation	Kumar (Test Lead)	
2)	Review	Hari Prasad (QA Analyst)	
3)	Approval	Vinod Rao (Project Manager)	

Example: Ecommerce Website Cross Browser Testing Test Plan

◆ **Test Plan Identifier:** Ecommerce Website Cross Browser Testing TP_1.0

◆ **References:** Project Plan, and Functional Specifications A1-C5.

◆ **Introduction :** This test plan for website cross browser testing supports the following objectives:

1. To define the tools to be used throughout the testing process.
2. To communicate to the responsible parties the items to be tested, set expectations, schedule and define environmental needs.
3. To define how tests will be conducted.

◆ Test Items

The system to be tested include the frontend customer facing website along with the backend ecommerce admin platform. These systems should be tested in Chrome 50.1, FireFox 45.0, Safari 9.1, and Internet Explorer 10 & 11. The system should be tested on both a Windows and Mac machine.

◆ **Features To Be Tested :** Features to be tested include the following

- As a shopper, logging into the website as a shopper
- As a shopper, navigating the store
- As a shopper, adding items to a shopping cart
- As a shopper, removing items from a shopping cart
- As a shopper, purchasing multiple units of the same item
- As a shopper, initiating a return
- As a shopper, contacting support
- As a shopper, completing an order cycle
- As a shopper, cancelling an order
- As a shopper, leaving a review
- As an admin, granting a refund
- As an admin, fulfilling an order
- As an admin, answering a support inquiry
- As an admin, moderating reviews
- As an admin, validating in stock/outofstock

◆ Features Not To Be Tested

Mobile purchasing through a mobile device will not be tested. Only desktop web browser functionality will be tested.

◆ Pass/Fail Criteria

All core functionality of the systems should function perform correctly. There must be no critical defects found and an end user must be able to complete a purchase cycle successfully and initiate a refund without any errors. 95% of all test cases should pass and no failed cases.

◆ Suspension Criteria

Testing should be paused immediately if either system experiences login issues or failure in any basic CRUD (Create, Read, Update and Delete) actions.

◆ **Test Deliverables**

Upon completion, the test run results will be saved in Test Lodge and the test manager should then run a report for all completed tests.

◆ **Testing Tasks:** The following activities must be completed:

- Test plan prepared.
- Functional specifications written and delivered to the testing team
- Environment should be ready for testing (test data, test logins, test payment information etc).
- Perform the tests.
- Prepare test summary report.

◆ **Environmental Needs**

The test site must be populated with test data including a variety of products with different prices. Test mode should be enabled for the backend ecommerce platform.

◆ **Responsibilities:** The Test Manager is responsible for facilitating the testing project, coordinating availability and schedule of testers and training them as needed. Each tester should understand the expectations on completion date and level of quality. The Test Manager should also communicate any risks to the team.

◆ **Staffing and Training Needs:** Testing should be done by two testers. Both testers should conduct testing on each system. The testers assigned should have basic knowledge of the e-commerce platform.

◆ **Schedule:** Testing will take place 4 weeks prior to the launch date. The first round of testing should be completed in 1 week.

◆ **Risks and Contingencies**

If the first round of testing is not completed within 1 week, it could delay bug fixes and final testing. If this happens, UAT would be pushed back and eventually affect the launch date. If the testers don't have a basic understanding of the ecommerce platform, testing could be delayed or not conducted properly.

◆ **Approvals**

The test manager and product manager both must agree on completion of the testing project and determine when it's ready to proceed to the next step.

🔊 **Preparing Traceability Matrix (RTM)**

Requirements Traceability Matrix is a document that maps and traces user requirement with test cases. i.e. shows relationship between Requirements and Test Cases. It traces the requirements given by the client. This helps to ensure that all the requirements will be covered in the testing process and related test cases have been developed. Functionality should not be missed. It is basically used to ensure that all requirements and Change Requests will be tested.

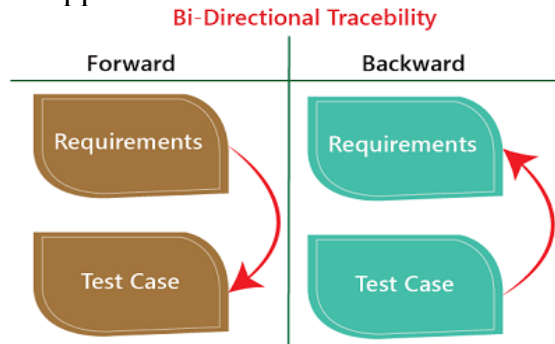
➤ **Requirement Traceability Matrix Parameters include**

- Requirement ID and Description
- Requirement Type
- Test Case ID and Description
- Test Status, Priority
- Bug ID etc.

➤ **Types of Traceability Test Matrix**

- ◆ **Forward traceability:** is used to ensure that every requirement is executed correctly in the application and tested systematically. This is to verify that whether the product developments are going in the right direction. It maps requirements to test cases.
- ◆ **Backward or reverse traceability:** is used to check that we are not increasing the scope of the project by adding features or functionality, code which are not mentioned in the original

requirements. It is used to ensure that existing project remains in the correct direction. Requirements are mapped into the backward direction to the test cases.



- ◆ **Bi-directional traceability (Forward+Backward):** It is a combination of forwarding and backward traceability matrix, which is used to make sure that all the business needs are executed in the test cases. It also evaluates the modification in the requirement which is occurring due to the bugs in the application.

➤ **How to prepare Requirement Traceability Matrix (RTM):**

1. Collect all the available requirement documents. eg. Business Requirement Document (BRD), Functional Requirement Document (FSD), Technical Requirement Document (TSD)
2. Allot a unique Requirement ID for each and every Requirement
3. Create Test Cases for each and every requirement and link Test Case IDs to the respective Requirement ID.

➤ **How to create Requirement Traceability Matrix**

Requirement Traceability Matrix through a banking project.

Test cases of Business Requirement Document for banking project:

The customer should be able to login to banking website with the correct password and userid while manager should be able to login to the website through customer login page.

BR#	Module Name	Applicable Roles	Description
B1	Login and Logout	Manager Customer	Customer: A customer can login using the login page Manager: A manager can login using the login page of customer. Post Login homepage will show different links based on role
B2	Enquiry	Customer	Customer: A customer can have multiple bank accounts. He can view balance of his accounts only Manager: A manager can view balance of all the customers who come under his supervision
B3	Fund Transfer	Manager Customer	Customer: A customer can have transfer funds from his "own" account to any destination account. Manager: A manager can transfer funds from any

Test cases of Technical Requirement Document (TRD) for banking project:

Login	
T92	User-ID must not be blank
T93	Password must not be blank
T94	If userid and password are valid. Login

Here is our TRD (Technical Requirement Document)

QA teams don't document the BRD and TRD. **Function Requirement Documents (FRD)** is similar to Technical Requirement Document but the process of creating RTM remains the same.

Step 1: Test Case "Verify Login" when correct ID and Password is entered then login successfully.

Test Case #	Test Case	Test Steps	Test Data	Expected Result
1	Verify Login	1) Go to Login Page 2) Enter UserID 3) Enter Password 4) Click Login	id= Guru99 pass= 1234	Login Successful

When correct password and id entered, it should login successfully

Step 2: Identify the Technical Requirement when test case is verifying. For our test case, the technical requirement is T94 is being verified.

T94 If userid and password are valid. Login

T94 is our technical requirement that verifies successful login

Step 3: this Technical Requirement (T94) in the Test Case.

TestCase #	TR #	Note the Technical Requirement in the test case	Test Steps	Test Data	Expected
1	T94	Verify Login	1) Go to Login Page 2) Enter UserID 3) Enter Password 4) Click Login	id= Guru99 pass= 1234	Login Successful

Step 4: Identify the Business Requirement for which TR (Technical Requirement-T94) is defined

BR#	Module Name	Applicable Roles	Description
B1	Login and Logout	Manager Customer	Customer: A customer can login using the login page Manager: A manager can login using the login page of customer. Post Login homepage will show different links based on role

Step 5: the BR (Business Requirement) in Test Case

TestCase #	BR #	TR #	Test Case	Test Steps	Test Data	Expected
1	B1	T94	Verify Login	1) Go to Login Page 2) Enter UserID 3) Enter Password 4) Click Login	id= Guru99 pass= 1234	Login Successful

Step 6: Do above for all Test Cases. Later Extract the First 3 Columns from your Test Suite. RTM in testing is Ready.

Business Requirement #	Technical Requirement #	Test Case ID
B1	T94	1
B2	T95	3
B3	T96	3
B4	T97	4

Requirement Traceability Matrix

➤ Advantage of Requirement Traceability Matrix

- It confirms 100% test coverage
- It highlights any requirements missing.
- It shows the overall defects with a focus on business requirements
- Gives Overview of all the requirements
- Shows how requirements are linked to Test Cases
- Easy to prepare
- No special tool is required

Example: Project-Online Flight Booking Application

➤ **Business Requirement Document (BRD):** Suppose Flight Booking Application it shows below 2 requirements

- **BR_1 Flight Booking Module:** allow user to book one or more tickets, one way or round way for future dates
- **BR_2 Payment Module:** to make payment for booked tickets via Credit / Debit Card or through Reward Points

➤ **Functional Specification Document (FSD):** This document is prepared by Technical team which further detailed business requirements into functional requirements that can be implemented in software.

Suppose above 2 business requirements in BRD have more detailed functional requirements:

BR_1 Reservation Module:

- **FR_1 : One Way Ticket booking :** allow user to book one way ticket
- **FR_2 Round Way Ticket :** allow user to book round way ticket

- **FR_3 Multicity Ticket booking** allow user to book one way or round way ticket for multiple cities

BR_2 Payment Module:

- **FR_4: By Credit Card:** allow user to make payment by Credit Cards
- **FR_5 By Debit Card:** allow user to make payment by Debit Cards
- **FR_6 By Reward Points:** allow user to make payment by Reward Points

Written some test cases or test scenarios for each functional requirement.

Test Case ID#	Test Case Description
TC#001	Verify if user is able to book one way ticket
TC#002	Verify if user is able to book multiple one way tickets
TC#003	Verify if user is able to book round way ticket
TC#004	Verify if user is able to book multiple round way tickets
TC#005	Verify if user is able to book one way ticket for multiple cities
TC#006	Verify if user is able to book round way ticket for multiple cities
TC#007	Verify if usre is able to pay by Master Card
TC#008	Verify if usre is able to pay by Visa Card
TC#009	Verify if usre is able to pay by Debit Cards
TC#010	Verify if usre is able to pay Fully by Reward Points
TC#011	Verify if usre is able to pay partially by Reward Points

Prepare simple Requirements Traceability Matrix (RTM) for above example it would like as below:

REQUIREMENTS TRACEABILITY MATRIX					
Project Name: Online Flight Booking Application					
Business Requirements Document BRD		Functional Requirements Document FSD			Test Case Document
Business Requirement ID#	Business Requirement / Business Use case	Functional Requirement ID#	Functional Requirement / Use Case	Priority	Test Case ID#
BR_1	Reservation Module	FR_1	One Way Ticket booking	High	TC#001 TC#002
		FR_2	Round Way Ticket		TC#003 TC#004
		FR_3	Multicity Ticket booking	High	TC#005 TC#006
BR_2	Payment Module	FR_4	By Credit Card	High	TC#007 TC#008
		FR_5	By Debit Card	High	TC#009
		FR_6	By Reward Points	Medium	TC#010 TC#011

You can also add Execution Status and Defects columns in RTM to view overall status of all requirements along with Test Cases.

🔊 Writing Test Execution Report

Test execution report contains the execution detail and the result of test execution. This is generally prepared and sends to higher management from testing team to show the state of the test execution and test progress. When we deliver the software to client, we shall send the full test execution detail and coverage in brief. Test execution is the process of executing the code and comparing the expected and actual results. Test execution report can be sent on a daily basis or weekly basis. The test execution report shows number of test cases passed, failed, blocked or not executed. It also list the bugs encountered during the test execution phase.

Element	Description
---------	-------------

Test item	Item, module or software that is been testing.
Scenario	Scenario of the test.
Responsible tester	Who tested the particular test item
Date of test execution	Date of the test executed.
Status	Status of test execution.
Test environment	Detail of test environment. Detail of hardware, software & tools.
Risk and constraint	Any risk that impact the test execution and any constrain.
Test execution summary	Provide detail of test execution , test type, defect rage etc.
Defect summary details	Defect count. Rate of open and closed defect. Severity and status wise defects.
Test cases detail	Planned and executed test cases. Status, result of execution and test case.
Test progress	Over all progress of test and which cycle the test execution is going on.

Different reports generally used in Testing. Before detailing out a test execution reports, let's find out the basic reports used by testers.

➤ Daily Status Report

Daily status reports that contains information about the different tasks performed, tasks executed, issues logged and difficulty faced by a tester. Every testing team member creates a daily status report and sends to their test manager at the end of each day.

Three primary testing reports are frequently used by testers:

- **Test Plan status report** – shows the current status of the test plan. This report notifies to team, whether any changes are being made to the existing test plan or any new version is introduced.
- **Test Documentation Status** – shows the status of the test cases, and test scenarios while the project is in the design phase.
- **Test Execution Status** – shows what the status of testing and how stable the overall application.

The basic components of test execution report can be broadly classified into 3 categories,

⇒ **Test Report Summary** - gives a brief overview of the overall execution status.

- The number of test cases planned within the cycle.
- The number of test cases executed within the cycle.
- The total number of test cases executed till date.
- Defects encountered within the cycle.
- Total defects encountered till date.
- Pass percentage and defect density.

The summary section helps the project manager and product owner get a quick overview of the current execution and analyze it to predict any blockage that might arise during the testing cycle

Figure: Test Summary Report

Project Name : Banking Application	
Overall progress of the QA cycle	
Total number of test cases	100
Number of testers	5
Test cycle duration	5 days
Status for 10/27/2017	
Number of test cases planned	20
Number of test cases executed	18
Number of test cases executed overall	78
Number of defects encountered today	2
Number of defect encountered so far	10
Number of critical defects- still open	3
Overall status	
Number of test cases planned	100
Number of test cases executed	78
Pass Percentage of the defects	98%
Defects density	2.5 per day
Critical defects percentage	20%

⇒ **Test Scenario/ Case status**

The test scenario/ test case status gives a detailed report of the status of the test scenarios.

It enlists the different scenarios and test cases executed during the testing cycle, along with their corresponding status. You can even find the name of the testers who executed the test cases along with a brief description of the issues which were encountered during test execution.

Module	Scenarios	Sub Levels	Complexity	Responsible tester	Date of Execution(Can be past, present or future date)	Status(Pass/Fail/Blocked/Not executed)	Defect ID- Brief description	Severity	Status
Blog	Blog entries	Content	Medium	Tester A	10/27/2017	Pass			
Blog	Social Widgets	Facebook	Complex	Tester B	10/27/2017	Pass			
		Twitter	Complex	Tester B	10/27/2017	Pass			
		Linkedin	Medium	Tester B	10/27/2017	Pass			
E-course	Course management	Add course	Complex	Tester A	10/27/2017	Pass			
		Make payment	Complex	Tester A	10/27/2017	Pass			
		Verify enrollment	Medium	Tester A	10/27/2017	Pass			
E-course	Resume ecourse	Verify e-course session management	Complex	Tester B	10/27/2017	Fail	#4573 - Session data doesn't persist on different tabs	1- High	Open
		Managing courses	Complex	Tester B	10/27/2017	Blocked	1028		
		Verify course content	Medium	Tester B	10/27/2017	Not executed			
E-course	Sharing course	Adding course	Complex	Tester X	10/27/2017	Pass			
		Sharing course	Complex	Tester X	10/27/2017	Pass			
		Selecting social widget	Medium	Tester X	10/27/2017	Fail	#5655 - Wdgets not	2- Medium	Assigned

⇒ Reported bugs

Any bugs which were reported during the test cycle are mentioned in this section.

Defect ID	Defect Description	Steps to reproduce	Severity	State	Created by	Assigned to
5655	Widget not visible on UI	1. Login to the site using the admin credentials 2. Click on E-course management. 3. Select e-course 4. Share e-course using social media widgets Expected result: The widgets should be visible on each e-course page Actual Result: Widget not visible	1- High	Open	Tester X	Dev representative

Figure: Reported bugs

✚ Test Summary Report

Test summary report is a document which contains summary of test activities and final test results. Client and Project management know the status of testing and Application Quality. This summary report is like to Test Closure phase of STLC.

Features of test summary report:

- Assessment of how well the Testing is performed during the testing process.
- Summarizes the results of all testing efforts.
- Client can evaluate the quality of the tested product and take a decision on the software release.
- Test managers prepare this document at the end of the testing project.

Test summary report will be different type for different types of testing.

- Suppose you are doing risk based testing your report should contain the test coverage and unresolved defects against the product risks.
- If you are doing requirements based testing should measure coverage against the requirements.

➤ Contents of a Test summary report:

Project Information	Test Objective	Test summary	Defect
• Project Name • Description	• Test Type • Purpose	• Test Passed • Test Failed • Test Blocked	• Description • Priority • Status

- Name of project
- Release No.

- Reference documents: Reference can be given of test planning document or test strategy document or project plan
- Target Audience: Who all will be the recipients of this report
- Test Cycle:

➤ **Testing Type / Level:** *Mention testing type / level like unit testing or system testing or integration testing or it may be a functional testing or performance testing etc*

➤ **Test Suite Data:**

- Number of test suites planned.
- No of test suites executed
- No of test suites could not be executed

➤ **Test Case Data:**

- Number of test cases planned.
- Number and percentage of test cases implemented.
- Number and percentage of test cases executed.
- Number and percentage of test cases passed.
- Number and percentage of test cases failed (total and by severity).

➤ **Defect Data:**

- Total open bugs
- New bugs found
- Open bugs of previous release
- Reopened bugs
- Bugs marked as "Not a Bug"
- Bugs marked as "Deferred"

➤ **Risks / open issues / support required:** Mention here all potential risks and current open issues with assignment of action items to specific stakeholder. Mention the details of support required, if any.

➤ Some other information can be part of test summary report:

- i) Testing Members
- ii) Objectives
- iii) Scope
- iv) Client Details
- v) Test Approach used
- vi) Test Results
- vii) Defect Report

Example: A Sample Test Summary Report

Introduction: It is for Internet Banking System Application Version 2.0, IBS (Internet Banking System) has Information, Personal banking, Corporate Banking, Master Data, User Management and Reports modules already. Now in this release some features added to Personal Banking Module and One New module “Small Business” added.

Test Items:

- Personal Banking
- New Business
- Reports

Reference documents:

- Test Plan
- Test Case documents
- Opened and Closed Defect Reports
- Metrics docs
- Review Reports.

Target Audience:

- Project Manager
- Release Team
- Maintenance Team and Customer (End Users).

+ Test Summary**a) Test Suite Information:**

- Test suites planned: 42
- Test suites implemented: 39
- Test suites executed: 38

b) Test Case Information:

- Test cases planned: 983
- Test cases implemented: 978
- Test cases executed: 967
- Test cases passed: 940 (Final)
- Test cases failed: 14 (Final)
- Test Cases pending: 3

c) Defect Report Information

- Defects found in comprehensive Testing: 148
- Test Cases selected for Regression Testing Cycle1: 320
- Defects found in Regression Testing Cycle1: 96
- Test Cases selected for Regression Testing Cycle 2: 213
- Defects found in Regression Testing Cycle 2: 42
- Test Cases selected for Regression Testing Cycle 3: 107
- Defects found in Regression Testing Cycle 3: 26
- Test Cases selected for Final Regression: 512
- Number of opened defects in this release: 14

Approvals

Name	Role	Responsibility	Date & Signature
Abhi Rao	Test Lead	Test Summary Report preparation	20-06-2012
Suyogkumar	QA Analyst	Review	25-06-2012
Murali	Project Manager	Approval	29-06-2012

Defect Life Cycle-

- Difference between Bug, Defect, Failure, Error
 - Defect Tracking and Reporting
 - Types of Bugs, Identifying the Bugs, Reporting the Bugs

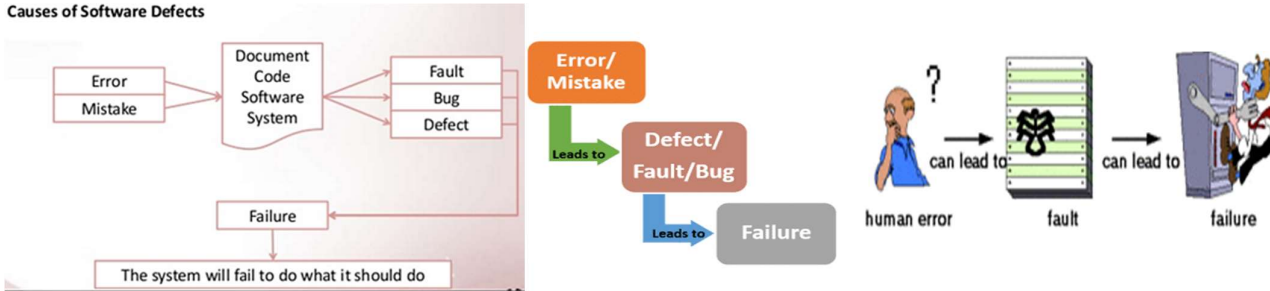
Introduction to automated testing-

- Install and configure selenium testing tool,
- Case study: Design test case for login page, Internet Banking Login, Online shopping.

Difference between Bug, Defect, Failure, Error

“A coding mistake is called an **error**. The error found by the tester is called a **defect**. A defect accepted by the development team is called **bug**. If the software does not meet the requirements, then it is **failure**.”

Causes of Software Defects



Error

An **Error** is a human mistake that produces an incorrect result. A mistake in the program leads to error due to incorrect loops or syntax. The mistake made by people. An Error appears not only due to the logical mistake in the code made by the developer but also anyone in the team can make mistakes during the different phases of software development.

Reasons of Error:

- Misinterpret or misunderstand of software requirements.
- Miscalculation of some values in the coding.
- If a developer is unable to successfully compile or run a program.
- Mistakes in design or requirement activities.
- Customer may provide insufficient or incorrect information.
- Difference between actual and expected results.

Types of Error

- User Interface Error: occurs during user interaction with the system. Such as missing or incorrect functionality of the system, no backup function etc.
- Syntax error: Misspelled words or grammatically incorrect sentences
- Calculation errors: occur due to bad logic, incorrect formulas, mismatched data type, etc.
- Flow control error: Occurs due to control of the program in wrong direction where software program acts unexpectedly i.e. infinite loop, syntax error during run-time, etc.
- Testing errors: Occur when implementing and executing the test process. i.e. bug scanning failure, inefficiency in reporting an error or defect.
- Hardware errors: Occurs due to configuration such as insufficient storage area for memory and no compatibility with the device.

Fault

Fault is a condition that causes a system to fail in performing its required function. Fault is incorrect step, process or data definition in a computer program which causes program acts an unexpected manner which is not in the desired output. It is an evidence of the error.

Bug

“A bug is an error, weakness, mistake or fault present in a coding that produces an incorrect or unexpected result that impact on the performance as well as functionality of the software”. Bugs arise when mistakes and errors made by people in either source code or its design. Typically bugs are found in unit testing by testers.

➤ Types of Bugs

- Algorithmic bugs such as division by zero.
- Logic bugs such as Infinite loops (a loop having no termination point).
- Resource bugs due to the use of uninitialized variables.
- Functional Errors
- Compilation and Runtime Errors
- Missing commands and Error handling errors

◎ Failure

Once the product is delivered to the customers and if the customer find any issues in the software it is called as condition of failure. Failure occurs when executing software code, that time software code fails to meet software requirements or functions due to occurrence of any defect i.e. Failure occurs when fault executes.

➤ Reasons for Failures:

- Human Error: Human interacting with the software by providing wrong or incomplete inputs.
- Environmental Conditions: Failures also happen by environmental conditions like a radiation burst, overheating, electronic fields, or pollution could cause faults in hardware or software.
- Users: Failures may also occur in the system if the user tries to execute operations with the intention of breaking the system.

Example1:

You are driving a car and you are on road while on driving now there is two way on the road

1) Left--> Mumbai

2) Right--> Delhi

You have to go to Delhi. you have to turn the steering to the right, but by mistake you turn the steering to the left, from that position that is called as "Error" because human interaction is there and now Fault is there till you will not reach the Mumbai, but when you reach Mumbai that is a final stage which is called "Failure" because you had to reach Delhi but now you are in Mumbai.

Example 2: this program is required to add two numbers.

<pre>#include<stdio.h> int main () { int value1, value2, ans; value1 = 5; value2 = 3; ans = value1 - value2; printf("The addition of 5 + 3 = %d.", ans); return 0; }</pre>	<pre>#include<stdio.h> int main () { int value1, value2, ans; value1 = 5; value2 = 3; ans = value1 - value2; // ----> Bug printf("The addition of 5 + 3 = %d.", ans); return 0; }</pre>
--	---

After compile and run program o/p is: **The addition of 5 + 3 = 2.**

So after output of program, we realize the program has failed. Because program displays the answer 2 instead of 8. We have detected a bug. Analyze the program what was the fault in the program.

We notice that a '-' sign present instead of '+' sign. So the fault in the program is the '-' sign. A mistake made by typing symbol '-' instead of '+' sign. Failure occur in execution of the program. We found bug. When tester testing the functionality of the program that time he realize output is faulty and raise a defect. Due to wrong result it is known error which resulted in the fault in the program and due to which the program failed to give the correct result.

◎ Defect

The variation between the actual results and expected results is known as defect. A defect is an error in coding or logic that causes a program to fail or produce incorrect result. A bugs introduced by programmer inside the code and corrected by the developer in development phase.

When a tester finds a defect they need to documented the details of the defect and communicate to the development team. A report contains the list of defects that have been found by

the tester, along with the details of the defects. This communication can take place through email by reporting the defect through a defect tracking tool like JIRA, Bugzilla etc.

➤ Reasons of Defects:

- User providing wrong and inaccurate inputs.
- Analyst interprets requirements incorrectly.
- Incorrect design.
- Error in coding.
- Data entry error.

➤ Types of Defects:

- **Missing Defects:** Occur due to client requirements not satisfy.

Example: if we take the above example and open the inbox link, we see that it is not there only, which means the feature is not developed only.

- **Wrong Defects:** Occur due to incorrect implementation of requirements. Which not mentioned in the requirement specification.

Example: Suppose a Gmail application, where we click on the "Inbox" link, and it navigates to the "Draft" page, this is happening because of the wrong coding which is done by the developer, that's why it is a wrong defect.

- **Extra Defect:** It is caused by a feature not in original list of client requirements but added later into the software.

Example: Suppose we have one application form wherein the **Name field**, the **First name**, and **Last name** textbox are needed to develop according to the client's requirement.

But, the developers also develop the "Middle name" textbox, which is not needed according to the client's requirements

➤ Benefits of using a defect report

- Defect report helps the development team (and testing team) to understand the defect
- It also enables the teams to track the defect to closure
- If the defect is not reported, which may lead to the defect not being fixed

➤ Defect categorization

Defects are classified from the QA team view as **Priority** and from the development view as **Severity** (complexity of code to fix it). These are two major classifications that play an important role in the timeframe and the amount of work that goes in to fix defects on priority basis. i.e. fixing those defects first that are highly critical.

These are mainly categorized into three types, based on level of severity, probability, and priority.

01. Severity Basis

Severity defines the defect's degree of impact. These reflect the degree or intensity of a particular defect and its impact on a software product or its working. These are further categorized into:

- Critical
- Major
- Minor
- Trivial

Probability Basis 02.

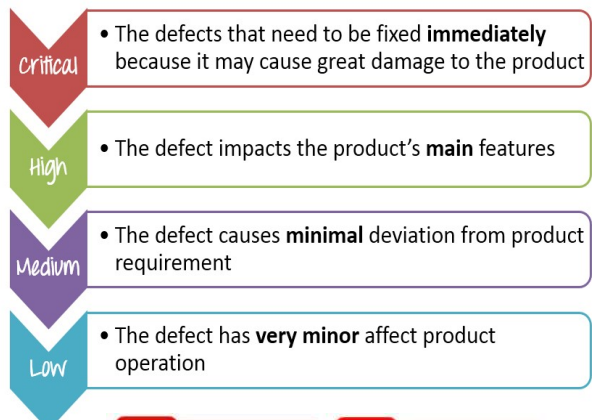
Defects are also categorized based on their probability to occur in the software or being encountered by the user. Based on this possibility, which is presented in terms of percentage, it divided into:

- High
- Medium
- Low

03. Priority Basis

Defects are also categorized on the basis of the business perspective. Here, defects with the highest priority are corrected first, based on the current need and demand of the business. These, like probability, are further categorized into:

- High
- Medium
- Low



P1 • Critical	S1 • Critical
P2 • High	S2 • Major
P3 • Medium	S3 • Moderate
P4 • Low	S4 • Low

Defect Priority Levels

Defect Severity Levels

No	Description	Priority	Explanation
1	The website performance is too slow	High	The performance bug can cause huge inconvenience to user.
2	The login function of the website does not work properly	Critical	Login is one of the main function of the banking website if this feature does not work, it is serious bugs
3	The GUI of the website does not display correctly on mobile devices	Medium	The defect affects the user who use Smartphone to view the website.
4	The website could not remember the user login session	High	This is a serious issue since the user will be able to login but not be able to perform any further transactions

Defect/Bug Report or Defect Tracking

Defect Report is a document where Test lead reported bug to developer. Using this documents developer identify exact position as well as location of the defect, so he can fix it easily.

- **Defect Id** – The Unique identification number assigned to the defect / bug
- **Defect Description** – Detailed description of the Defect
- **Date Raised/Reported on** – Date when the defect is reported.
- **Detected By/Reported by** – Name of the person who reported defect.
- **Assigned To/Fixed by** - Name/ID of the developer is assigned to analyze/ fix the defect.
- **Severity** – Impact of the defect or bug. This can be minor, major or critical.
- **Priority** – This can be low, medium or high. It is the priority or importance of the defect.
- **Status** – Track the status of the defect. The status can be New, Assigned, Open, Rejected, Fixed, Differed, Reopened, Verified or Closed.
- **Version** – Version of the application in which defect was found.
- **Closed On** – Date when the defect is closed

Bug Report Template

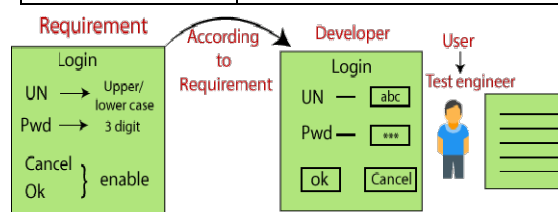
A bug report is a document prepared during the process of testing that offers all the necessary information regarding the various defects, errors, bugs, and other discrepancies found in the software product. This report follows a standard format to deliver accurate and concise information, which includes the following details:

- Bug Identifier
- Project.
- Build Number.
- Path.
- Severity.
- Priority.
- Assigned To.
- Reported by.
- Reported on.
- Bug Status.
- Source URL.
- Steps to Reproduce.
- Actual Result.
- Expected Result.
- Test Environment.
- Bug description.
- Attachments.



Example:

Defect ID	Boo12
Module	Login
Test case name	Gmail_login_Compose mail
Reported by	Kundan Test engineer
Version	3.0
Status	assigned
Date	23-02-2020
Assign to	Malvika developer
Severity	critical
Priority	P2
Platform	Window XP, internet explorer7.0
Build no.	B02
Test data	Username=xyz, password= 123
Brief description	
Navigation steps	▪ Login Gmail application ▪ Compose mail and confirmation message should be displayed
Expected result	Should also be in the inbox.
Actual result	Mail not found in inbox

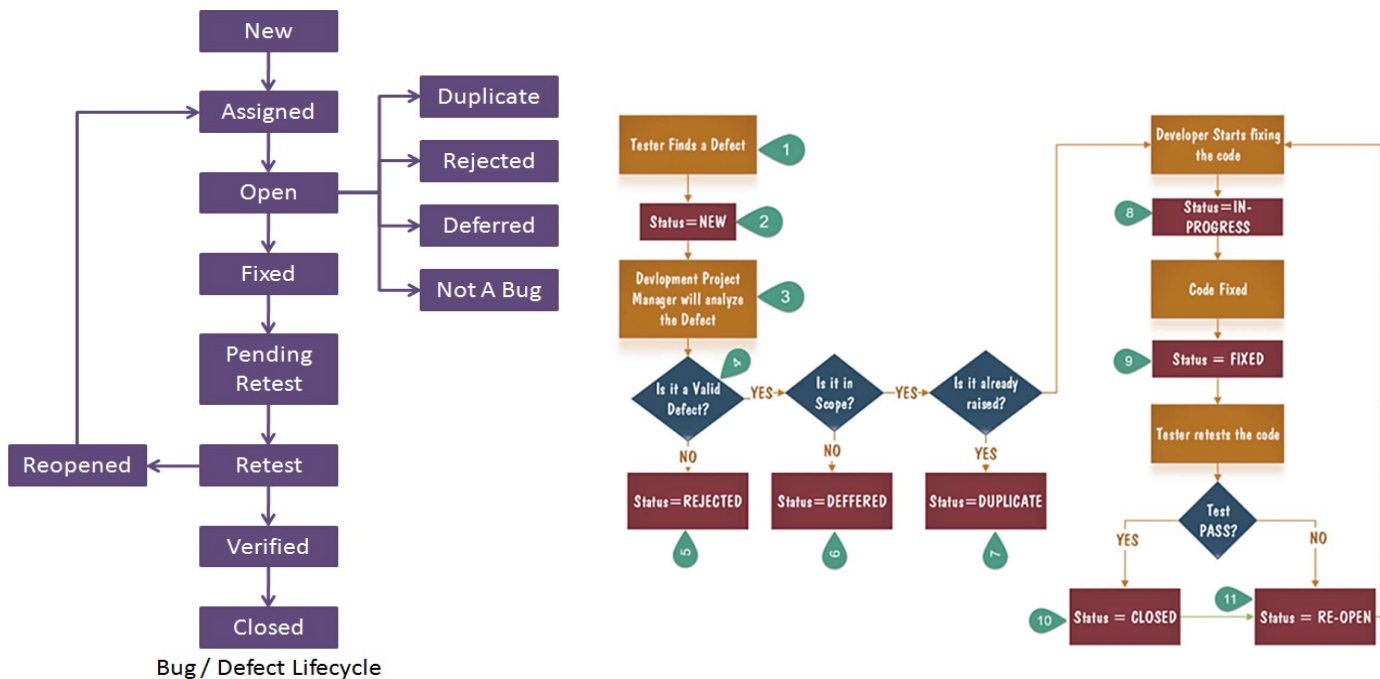


Defect/Bug Life Cycle

No software exists without a defect or bug. Removing bugs from the software depends upon the efficiency of testing on the software. A defect is the difference between the expected result and the actual result. The cost of finding and correcting of defects is most expensive software development activities.

Defect life cycle is a process of report and resolve also track the status of bugs. The goal of defect life cycle or bug life cycle is to resolve and test the bug or defect and follow the process till defect or bug exists. Defect or bug can be raised at any point during testing and could be reported to the relevant development team to fix it. Defect fixing process systematic and efficient.

Defect life cycle is starts when defect is found and ends when a defect is closed, after ensuring it's not reproduced. Defect goes through number of states.



New:	When a bug is detected for the first time in the software. It is assigned a status NEW.
Assigned:	A Reported bug testing team will approved the defect is genuine and assigned to the corresponding developer for fixing.
Open:	The developer starts analyzing and works on the defect fix
Fixed:	When a bug is resolved by the developer after making necessary changes in code
Pending retest:	After the developer has <u>fixed up the defect</u> , they send an application to the tester for testing. Retesting remains pending from the testers end
Retest:	After the defect is fixed by developer, it is assigned to the tester for retest the application again to see if the bug is reproduced again.
Verified:	If tester does not find any type of bug in the software, then he approves that the bug is fixed and changes the status to "Verified".
Reopen:	If the bug still exists even after the bug is fixed by the developer, the tester changes the status to "Reopened". The bug traverses the life cycle once again.
Closed:	Once the bug is fixed, it is tested by the tester. If the tester feels that the bug no longer exists in the software, he changes the status of the bug to "Closed".
Duplicate:	If the bug was reported earlier and again it is reported then the status is "Duplicate".
Rejected:	If the developer feels that the bug is not genuine, he rejects the bug. Then status is changed to "Rejected".
Deferred:	If the reported bug is not a higher priority and it can be fixed in the next release then the defect is marked as "Deferred" and will be fixed in upcoming releases.
Not a bug:	If testing team finds out that the defect is not harmful to the application then the status assigned to "Not a bug".

5. Reduces the test execution time and cost.
6. Minimizing Human Interaction
7. Automation scripts can run at any time and any number of times.
8. Reusability of scripts on other applications.
9. Easy to test Stress, Performance and Load
10. Saves manual efforts.

➤ Disadvantages of Automation Testing

1. Debugging the test script is a major issue. It may lead to consequences if any error is present in the test script
2. Proficiency is required to write automation test scripts
3. Test maintenance is costly
4. Maintenance of test data is difficult.
5. Testers should have basic programming knowledge
6. Testers should have experience in writing automation test scripts.

➤ Which Test Cases can be automated?

Test Cases should be automated if the following conditions are met.

1. Tests that are used repeatedly
2. High-Risk test cases
3. Tests that tend to cause human error
4. Test Cases that are impossible to perform manually
5. Tests that take lot of time to do manual testing
6. Time consuming test scenarios
7. Regression test suites
8. GUI items
9. Database connections.

➤ Automated Testing Process:

Following steps are followed in an Automation Process

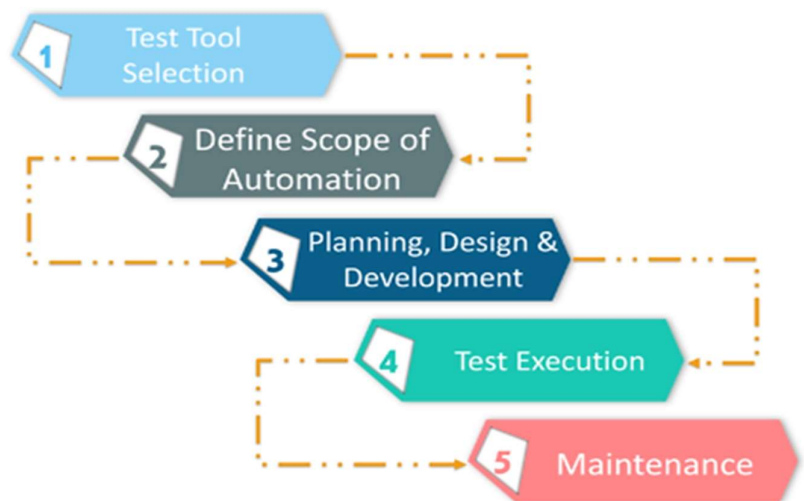
Step 1) Test Tool Selection

Step 2) Define scope of Automation

Step 3) Planning, Design and Development

Step 4) Test Execution

Step 5) Maintenance



✂ Install and configure selenium testing tool

The key-step to begin with [Selenium](#) is to install and get familiar with it. In order to help you out, I have come up with an article on Selenium Installation. Through the medium of this article, I will guide you through the simple steps to configure and install selenium on your system.

The process of installing Selenium involves 3 steps, namely:

1. [Install Java](#)
2. [Install Eclipse IDE](#)
3. [Install Selenium WebDriver](#)

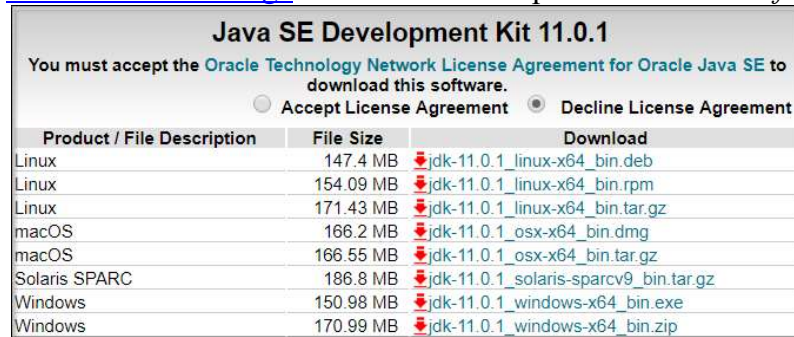
Let's get started.

Selenium supports various programming languages like [Java](#), [Python](#), C#, Perl, Ruby etc. Java is one of the most widely used languages for Selenium. Let's begin and understand the process of Java Installation. The very first step, needed for Java installation is to configure Java environment so that later on Java codes can be used to drive the selenium.

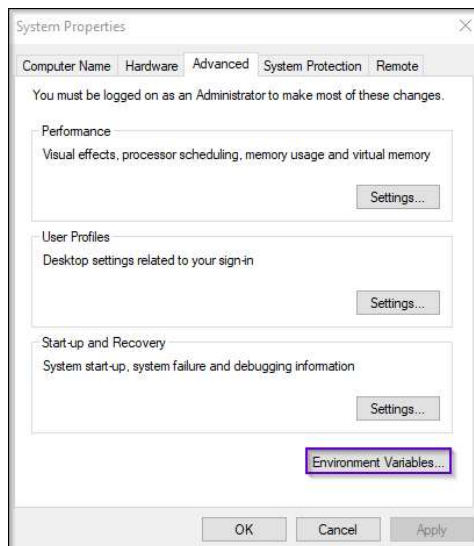
Install Java

Follow below steps to complete your [Java](#) installation.

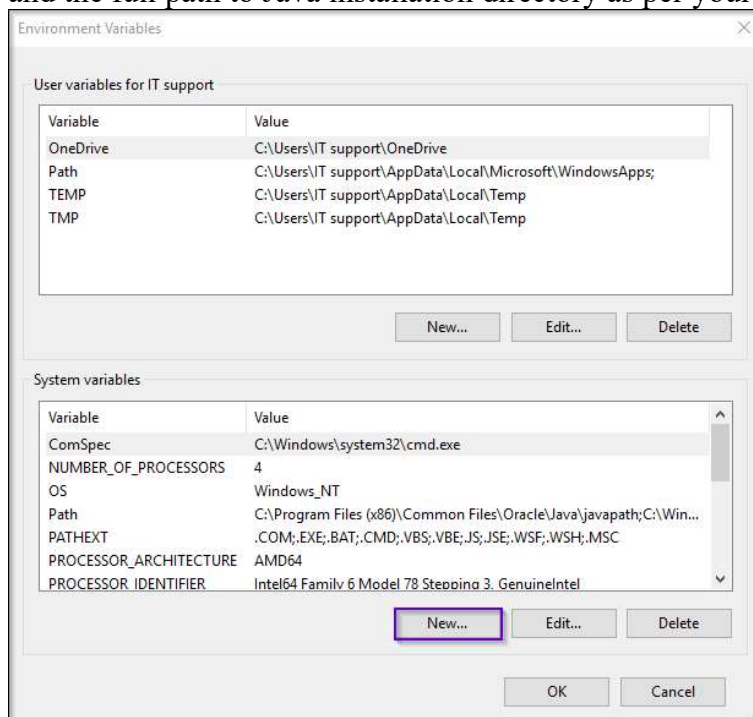
- Go to the [Java Downloads Page](#) and click on the option for *Java Platform (JDK)*.



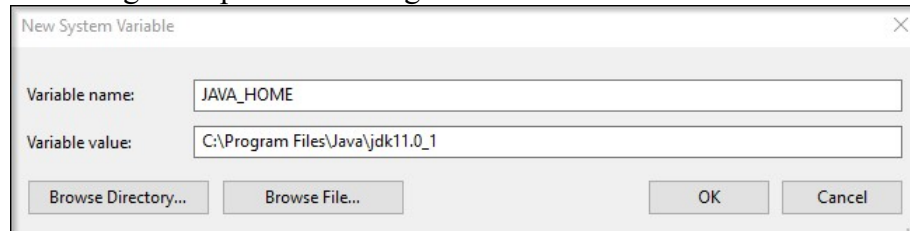
- In the next page, select the Accept License Agreement radio button, accept it and click the download link against your matching system configuration.
- You can run the installer once the download is over and follow onscreen instructions.
 - Go to start and search for 'System'
 - Click on 'System'
 - Click on 'Advanced system settings'
 - Click on 'Environment Variables' under 'Advanced' tab as shown below:



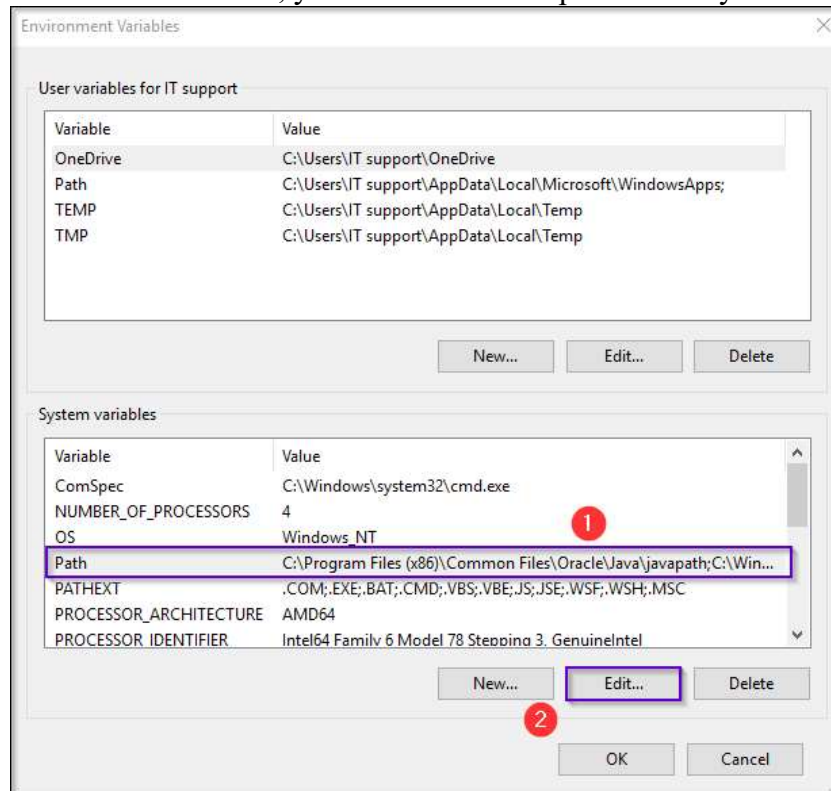
- Next, under system variables choose new and enter the variable name as 'JAVA_HOME' and the full path to Java installation directory as per your system as shown below:



Below figure depicts the configuration of environment variable name and value.



- Next thing that you have to do is to configure your environment variables. Let's see how to do that. Here, you have to edit the path of the system variable as shown below.



- Under 'Variable value', at the end of the line, enter the following path – `%JAVA_HOME%bin;`
Now, you can click 'OK' and you are done.
- Now to cross-check the installation, just run following command in cmd – `java -version`. It should display the installed version of Java in your system.

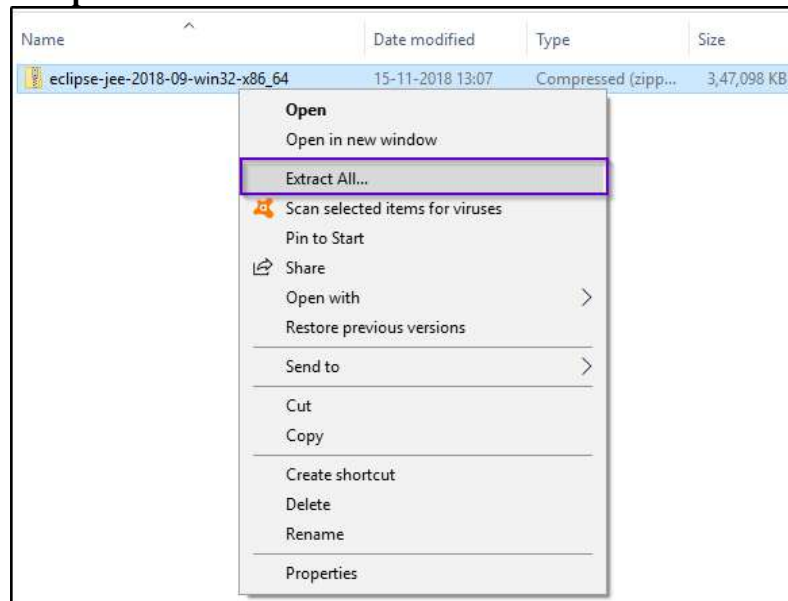
Install Eclipse

Follow the below steps to configure Eclipse on your system:

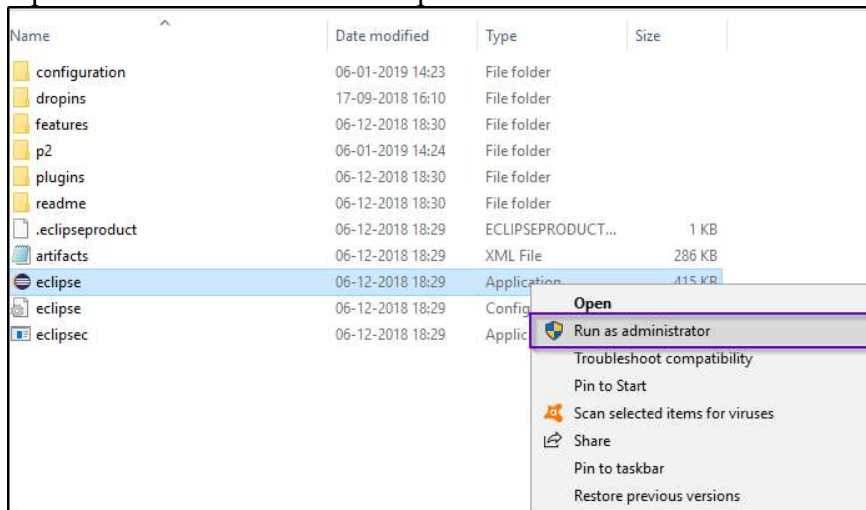
- Navigate to the following URL – <https://www.eclipse.org/downloads/> and select the download link depending on your system architecture – (32 Bit or 64 Bit) and download it.



- Once the download is over, extract the zipped file and save it to any directory. The root folder is the eclipse.



- Open the folder and launch eclipse.exe.

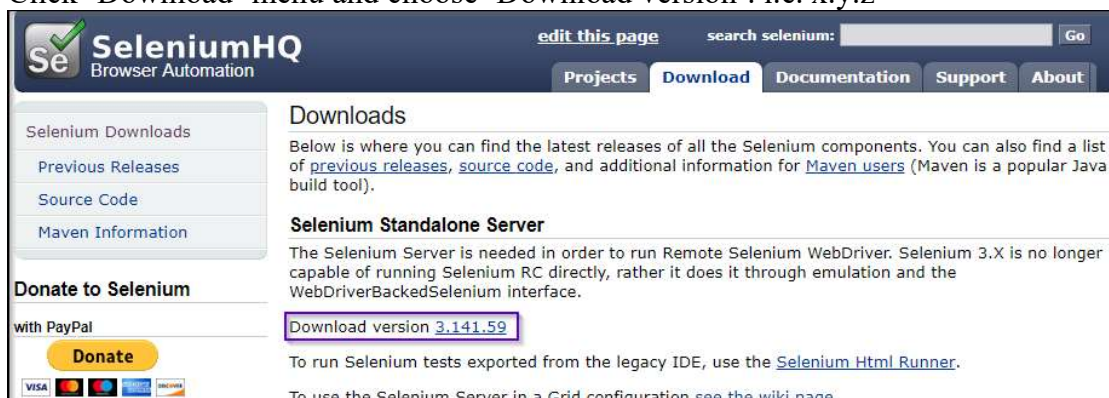


Now, the last step is to install [Selenium](#).

Install selenium and configure it in your system.

Install Selenium Web Driver

- Open the browser and navigate to <http://www.seleniumhq.org>.
- Click 'Download' menu and choose 'Download version'. i.e. x.y.z



- The download should get started for 'selenium-server-standalone-x.y.z.jar'. Save this JAR in "C:Selenium".
- Next, you have to download [Selenium](#) Java Client. In the same downloads page, scroll down and you will find a section called Selenium Client and WebDriver **Language**.

Now, click on Download link under Java as shown below.

Selenium Client & WebDriver Language Bindings

In order to create scripts that interact with the Selenium Server (Selenium RC, Selenium Remote WebDriver) or create local Selenium WebDriver scripts, you need to make use of language-specific client drivers. These languages include both 1.x and 2.x style clients.

While language bindings for [other languages exist](#), these are the core ones that are supported by the main project hosted on GitHub.

Language	Client Version	Release Date	Download	Change log	Javadoc
Java	3.141.59	2018-11-14	Download	Change log	Javadoc
C#	3.14.0	2018-08-02	Download	Change log	API docs
Ruby	3.14.0	2018-08-03	Download	Change log	API docs
Python	3.14.0	2018-08-02	Download	Change log	API docs
Javascript (Node)	4.0.0-alpha.1	2018-01-13	Download	Change log	API docs

- Extract the zip file and save it in your selenium folder.
- Next, you have to download Chrome Driver for Google Chrome and Gecko Driver if you are using Mozilla Firefox.
- In the same downloads page, scroll further down and you will find third-party drivers section. There you will find options for downloading gecko driver and chrome driver as shown in the below figure.

Third Party Drivers, Bindings, and Plugins

Selenium can be extended through the use of plugins. Here are a number of plugins created and maintained by third parties. For more information on how to create your own plugin or have it listed, consult the docs.

Please note that these plugins are not supported, maintained, hosted, or endorsed by the Selenium project. In addition, be advised that the plugins listed below are not necessarily licensed under the Apache License v.2.0. Some of the plugins are available under another free and open source software license; others are only available under a proprietary license. Any questions about plugins and their license of distribution need to be raised with their respective developer(s).

Third Party Browser Drivers NOT DEVELOPED by seleniumhq

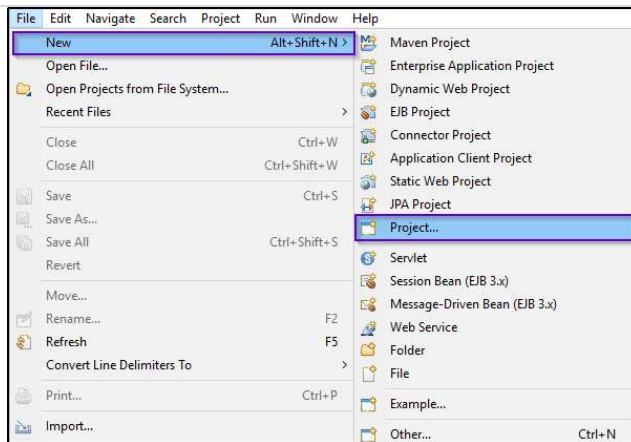
Browser	Latest	change log	issue tracker	Implementation Status
Mozilla GeckoDriver	latest	change log	issue tracker	Implementation Status
Google Chrome Driver	latest	change log	issue tracker	selenium wiki page
Opera	latest		issue tracker	selenium wiki page
Microsoft Edge Driver			issue tracker	Implementation Status

- In the next page, you will find the latest version available. You can choose that and download based on your system configuration. You can refer the below image for your OS configuration.

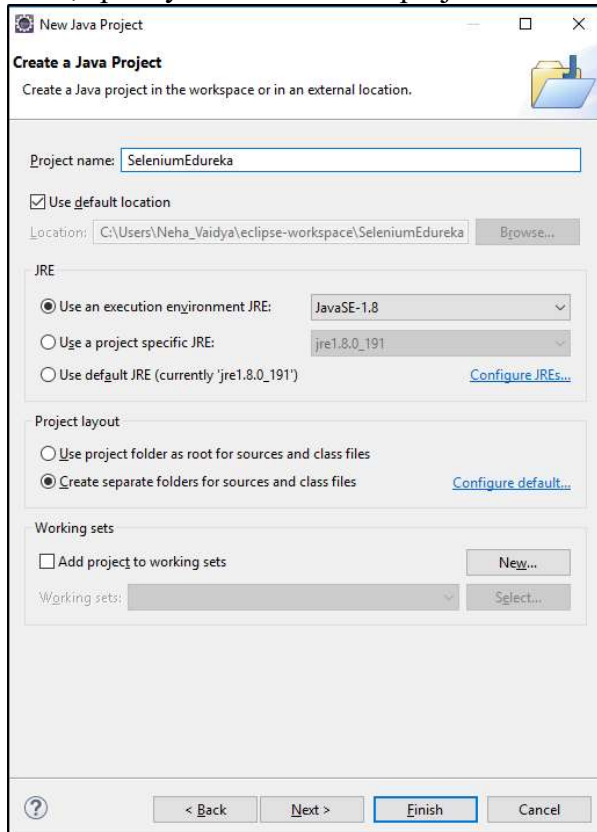
Index of /2.45/

Name	Last modified	Size	ETag
Parent Directory	-	-	-
chromedriver_linux64.zip	2018-12-10 23:20:22	5.14MB	61c3484abc908c543b823cf42f58fc5d
chromedriver_mac64.zip	2018-12-11 04:12:22	6.63MB	9f6b235f30b6ea4579612273595c8301
chromedriver_win32.zip	2018-12-11 01:10:40	4.39MB	6ee964d7172fb8bcd8538ccb430b53ed
notes.txt	2018-12-11 05:17:39	0.02MB	d98475f7fad0a16aa0543c01defa518c

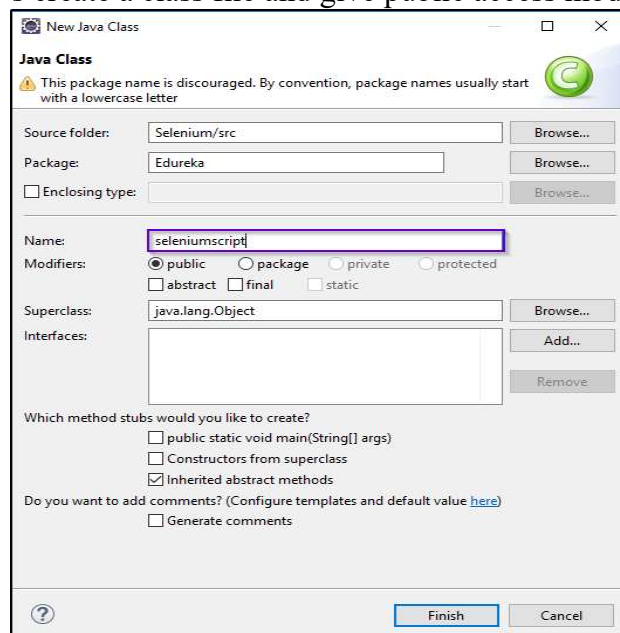
- You can simply download the driver, extract the zip file and save it in any location. Similarly, you can download the latest version of [gecko driver](#) and save it if you are using the Mozilla browser.
- Next, you can simply create a new project in Eclipse IDE and add all the external selenium jar files from selenium lib directory.
 - First, launch Eclipse and create new project as shown below.



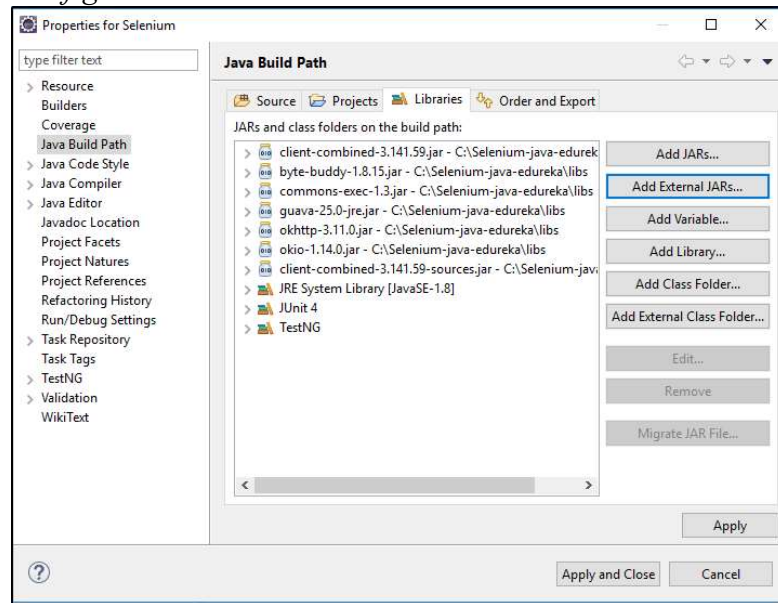
Next, specify the name of the project and create a class.



Let's create a class file and give public access modifier.



- Next, you should add referenced libraries to your project. Click on your *project* -> *Build Path* -> *Configure Build Path* and add external JARs as shown in the figure.



- Now you are all set to execute the first Selenium program. You can run the below script and check the output.

```
package edureka;
import java.util.concurrent.TimeUnit;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
public class FirstSeleniumScript {
public static void main(String[] args) throws InterruptedException {
System.setProperty("webdriver.chrome.driver", "C:\\Selenium-java-edureka\\chromedriver_win32\\chromedriver.exe");
WebDriver driver = new ChromeDriver();
driver.manage().window().maximize();
driver.manage().deleteAllCookies();
driver.manage().timeouts().pageLoadTimeout(40, TimeUnit.SECONDS);
driver.manage().timeouts().implicitlyWait(30, TimeUnit.SECONDS);
driver.get("https://login.yahoo.com/");
driver.findElement(By.xpath("//input[@id='login-username']")).sendKeys("edureka@yahoo.com");
}
}
```

Write a Test case for Facebook Login page Functionality using TestNG.

```
package FacebookLogin;
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.testng.Assert;
import org.testng.annotations.Test;

public class FacebookTest
```

```

{
    @Test
    public void verifyFacebookLoginPage() throws InterruptedException
    {
        System.out.println("Hello Login into Facebook");
        System.setProperty("webdriver.chrome.driver", "C:\\browserdriver\\chromedriver.exe");
        ChromeDriver driver=new ChromeDriver();
        driver.get("https://www.facebook.com");
        driver.manage().window().maximize();
        WebElement LoBtn=driver.findElement(By.xpath("//button[text()='Log In']"));
        String LoginButton=LoBtn.getText();
        Assert.assertEquals(LoginButton, "LogIn");
        Thread.sleep(9000);
        System.out.println("Close Browser");
        driver.close();
    }
    @Test
    public void verifyFBLoginWithValidCredentials() throws InterruptedException
    {
        System.out.println("Hello Login into Facebook");
        System.setProperty("webdriver.chrome.driver", "C:\\browserdriver\\chromedriver.exe");
        ChromeDriver driver=new ChromeDriver();
        driver.get("https://www.facebook.com");
        driver.manage().window().maximize();
        driver.findElement(By.xpath("//input[@class='inputtext_55r1_6luy']")).sendKeys("sangolacollege@gmail.com");
        driver.findElement(By.xpath("//*[@id='pass']")).sendKeys("sangola");
        driver.findElement(By.xpath("//button[text()='Log In']")).click();
        Thread.sleep(17000);
        System.out.println("Close Browser");
        driver.close();
    }
}

```

TestNG Testing Framework

1. Introduction to TestNG

TestNG (Test Next Generation) is an open-source automated testing framework inspired by JUnit and NUnit. It is designed to be more powerful and flexible, covering a wider range of testing categories including unit, functional, end-to-end, and integration testing.

Key Advantages:

- **Flexible Configuration:** Uses annotations to manage the test lifecycle.
 - **Powerful Execution Model:** Supports parallel testing, grouping, and sequencing.
 - **XML-based Suites:** Allows you to define specific test runs via a `testng.xml` file.
 - **Built-in Reporting:** Generates detailed HTML and XML reports automatically.
 - **No Main Method:** Eliminates the need for a `public static void main` method to trigger tests.
-

2. TestNG Annotations

Annotations are the heart of TestNG. They act as instructions for the compiler to determine the execution flow.

Annotation	Execution Level / Description
@BeforeSuite	Runs once before all tests in the entire suite.
@BeforeTest	Runs before any test methods inside the <code><test></code> tag in the XML file.
@BeforeClass	Runs once before the first test method in the current class.
@BeforeMethod	Runs before every individual <code>@Test</code> method (e.g., launching a browser).
@Test	The actual test case. Logic to be validated goes here.
@AfterMethod	Runs after every individual <code>@Test</code> method (e.g., closing a browser).
@AfterClass	Runs once after all test methods in the current class are finished.
@AfterSuite	Runs once after all tests in the entire suite have finished.

3. Core Methods & Attributes

You can control the behavior of your `@Test` methods using various attributes:

- **Priority:** Defines the order of execution. Lower values run first.
 - *Example:* `@Test(priority = 1)`
 - **DependsOnMethods:** Ensures a test only runs if a specific previous test passed.
 - *Example:* `@Test(dependsOnMethods = {"login"})`
 - **InvocationCount:** Executes the same test multiple times.
 - *Example:* `@Test(invocationCount = 5)`
 - **Enabled:** If set to `false`, TestNG will ignore/skip that specific test.
 - **Groups:** Categorizes tests (e.g., "Smoke", "Regression") for selective execution.
-

4. Creating an Automated Test Case

A standard TestNG class uses annotations to set up the environment, perform the test, and clean up afterward.

Sample Script:

```
Java
import org.testng.annotations.*;
import org.testng.Assert;

public class LoginAutomation {
    @BeforeMethod
    public void setup() {
        System.out.println(""Step 1: Launching Browser..."");
    }

    @Test(priority = 1)
    public void verifyTitle() {
        String expectedTitle = ""Dashboard";
        String actualTitle = ""Dashboard";
        Assert.assertEquals(actualTitle, expectedTitle);
        System.out.println(""Step 2: Title Verification Passed."");
    }

    @AfterMethod
    public void tearDown() {
        System.out.println(""Step 3: Closing Browser."");
    }
}
```

5. Automated Reporting

TestNG provides automated reporting without requiring extra code.

How to Access Reports:

1. **Execute the Test:** Run your code via your IDE (Eclipse/IntelliJ) or `testng.xml`.
2. **Refresh Project:** Right-click your project folder and select "Refresh".
3. **Locate Folder:** Find the folder named `test-output`.

4. View Reports:

- **index.html:** A detailed report showing pass/fail status, time taken, and groups.
- **emailable-report.html:** A simplified, single-page report designed for sharing with stakeholders via email.

Another Example of TestNG which generates a test report of youtube testing

FileName- M.java

```
import org.openqa.selenium.By;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.*;
import org.testng.Assert;

public class M {
    @Test
    public void run()
    {
        try
        {
            System.setProperty("webdriver.chrome.driver", "D:\\Selenium\\chromedriver-win64\\chromedriver-
            win64\\chromedriver.exe");
            ChromeDriver obj=new ChromeDriver();
            obj.get("https://www.youtube.com/");
            Thread.sleep(1000);
            obj.findElement(By.xpath("/html/body/ytd-app/div[1]/div[2]/ytd-
            masthead/div[4]/div[2]/yt-searchbox/div[1]/div/form/input")).sendKeys("sangolacollege");
            obj.findElement(By.xpath("/html/body/ytd-app/div[1]/div[2]/ytd-
            masthead/div[4]/div[2]/yt-searchbox/div[1]/button/span/span/div")).click();
        }
        catch(Exception e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```